Problem Chosen	Team Control Number
A	**295**

自来水厂水质预测与评估

摘要

本文针对某自来水厂 2025 年度 12 个月连续监测数据（每 2 小时采样，含原水、工艺过程、清水池三级共 18 个水质与运行指标），系统建立了从特征筛选、时滞建模、物理信息混合预测到风险评价的完整数学模型链。

首先，对原始数据进行多格式日期解析、插值填补与孤立森林–IQR 联合异常检测，并构造滞后、差分、交互三类时序特征。针对问题一（出厂水浊度影响因子筛选与预测），采用 Spearman 秩相关、LASSO 稀疏回归和 XGBoost+SHAP 值三种互补方法集成排名，筛选出月份、FILT.NTU 和 RW_FLOW 为最具一致性的核心驱动因子；ADF 检验确认 NTU 序列非平稳（p 值 =1.000），据此构建 Bootstrap 重抽样全局置信区间（均值 0.4531，95% CI [0.4335, 0.4739]）和 30 天滚动窗口预测，联合异常检测率为 4.95%。

针对问题二（滤后水浊度动态时滞建模），利用带预白化的互相关函数客观识别各输入变量对 FILT.NTU 的时间滞后——RW_NTU 滞后 9 步（18 h）、RW_FLOW 滞后 10 步（20 h）、ALUM 和 RW_PH 即时响应（0 步），辨识结果与混凝—沉淀—过滤全流程物理路径高度吻合。基于识别出的时滞参数构建 NARX 神经网络，测试集 RMSE=0.0552，$R^2 = 0.577$，显著优于持续性基线。

针对问题三（出厂水浊度混合动态预测），基于质量守恒原理推导清水池 CSTR 模型：$\mathrm{NTU}(t) = \alpha \cdot \mathrm{NTU}(t - \Delta t) + (1 - \alpha) \cdot \mathrm{FILT\_NTU}(t)$，平滑因子 $\alpha = 0.607$。将此物理约束嵌入 2 层 GRU 神经网络（隐维 64）的损失函数，构造物理信息混合损失 $\mathcal{L} = \mathrm{MSE} + \lambda_{\mathrm{phys}} \cdot \mathcal{L}_{\mathrm{CSTR}}$，实现物理规律对数据驱动模型的解空间约束。模型在 42 轮训练后早停收敛，2–12 h 多步预测 RMSE 稳定在约 0.28，显著优于持续性基线（RMSE 约 73.0），物理约束有效抑制了误差累积。敏感性分析确认 FILT_NTU 与 RW_NTU 为 12 h 预测的关键驱动变量。

针对问题四（水质风险评价），以 NTU≤1 为硬性安全标准，构建超标幅度、超标时长、连续超标时长和累积超标量四维指标体系，采用熵权法客观赋权（持续异常相关指标权重各 0.318，峰值权重仅 0.048），结合硬触发规则与百分位分级策略实现四级风险分类。2026 年评估结果为：安全 5 天（83.3%）、低风险 1 天（16.7%），整体水质状况良好。

关键词： 自来水水质预测；特征筛选；互相关时滞分析；NARX 动态模型；CSTR 物理约束；GRU 混合预测；熵权法水质风险评价

目录

1 问题背景与重述

1.1 问题背景

随着城市化与工业化进程的加速，优质自来水已成为城市竞争力的关键基础设施之一。因此，建立可靠的水质预测与评估数学模型，对保障饮用水安全、实现工艺智能调控具有重要的理论价值与现实意义。

某自来水厂提供了连续 15 个月的运行监测数据，每天记录 12 次（每 2 小时一次），涵盖原水水质指标（水位、泵频率、流量、浊度、色度、pH 值等）、工艺过程参数（滤后水浊度、矾投加量/流量）、清水池水质指标（水位、pH、浊度、色度、余氯等）及设备运行状态。该数据具有高时间分辨率、多变量耦合、非线性关系及显著时滞等特征，能够反映水厂在不同原水条件、气候状况及运行策略下的动态响应规律。

1.2 问题重述

基于上述背景与数据，本文需依次解决以下四个问题：

问题一：主要影响因素筛选与浊度预测。需采用适当的定量分析方法，从众多监测变量中筛选出影响自来水浊度（NTU）的主要因素，解释各因素的影响程度与作用方向，并建立主要影响因素之间的函数关系。在此基础上，预测附件 2 中 2026 年 2 月 1 日、2 月 10 日、2 月 20 日的水浊度值，检验模型预测效果，并以 Excel 表格形式给出预测结果。

问题二：滤后水浊度的动态建模与时滞分析。滤后水浊度（FILT. NTU）是衡量混凝—沉淀—过滤效果的核心指标，主要受原水浊度（R/W NTU）、原水 pH（R/W PH）、矾投加量（ALUM）及原水流量（R/W FLOW）等因素影响，且各输入变量对滤后水浊度的影响存在不同的时间滞后。需建立一个动态数学模型，刻画上述输入变量与滤后水浊度之间的定量关系，给出各变量的时滞参数，并对模型进行参数估计与验证，提供模型在自选数据上的拟合精度（如 RMSE、R^2）。

问题三：出厂水浊度的混合动态预测。出厂水浊度达标是水厂运行的核心目标，但由于从原水到出厂水需流经多个工艺环节，直接预测未来 6 小时乃至 12 小时后的出厂水质极具挑战。需结合质量守恒原理（如清水池的水力停留时间分布）与数据驱动方法（如 LSTM、GRU 或状态空间模型），构建一种混合动态模型，实现未来 1 至 12 小时出厂水浊度（NTU）的预测，并给出 2026 年 2 月 1 日、2 月 10 日、2 月 20 日 7 时至 19 时的逐时预测结果。此外，需分析不同输入变量（尤其是原水水质突变与矾投加量调整）对预测结果的敏感性。

问题四：水质风险评价体系构建。以水浊度（NTU）为核心指标，综合考量超标幅度与异常持续时长，建立水质风险评价体系，将 2026 年近 3 个月的水质状况划分为安全、低风险、中风险、高风险四个等级，并统计各等级天数占比，同时给出 3 月份逐日的具体分类结果。评价须以浊度限值（≤ 1 NTU）作为超标与风险判定的统一硬性约束。

2 模型假设

1. **数据质量假设**。经清洗与插值处理后的 2025 年监测数据能够真实反映水厂运行状态，异常值（孤立森林 + IQR 联合检测，异常率约 4.95%）可被辨识且不影响模型推断；2026 年水质变化趋势与历史规律基本一致。
2. **过程物理假设**。清水池内部水力混合充分，可近似为 CSTR 串联模型，出口浊度满足一阶指数平滑规律：$\mathrm{NTU}(t) = \alpha \cdot \mathrm{NTU}(t - \Delta t) + (1 - \alpha) \cdot \mathrm{FILT\_NTU}(t - \delta)$，其中 α 由水力停留时间决定；各工艺环节间存在客观可测的水力传输时滞。
3. **模型适用假设**。水厂主要处理单元在研究时段内运行状态相对稳定，无重大工艺改造或停运事件，水质波动主要由原水条件变化与药剂投加策略调整引起。

3 问题分析

问题一分析（浊度影响因素筛选与预测）。可通过 Bootstrap 重抽样构建全局置信区间（均值 0.4531，95% CI [0.4335, 0.4739]；中位数 0.3000，95% CI [0.2800, 0.3100]），同时引入 30 天滚动窗口均值捕捉局部趋势。在特征筛选层面，Spearman 秩相关、LASSO 稀疏回归与 XGBoost+SHAP 值三者集成分析表明，**月份**（Spearman $\rho = +0.47$）、**FILT_NTU**（SHAP=0.0375）、**RW_FLOW**（Spearman $\rho = +0.39$）是最具一致性的关键影响因子。每月均值（如 2 月 $\bar{x} = 0.6246$）显著高于年均值，提示模型须纳入季节效应。

问题二分析（滤后水浊度动态时滞建模）。互相关函数分析揭示了各输入变量对 FILT.NTU 的差异化时滞特征：RW_NTU 影响滞后 9 步（18 h），这与原水经混凝—沉淀—过滤全流程到达滤后检测点的物理路径一致；ALUM 和 RW_PH 的 CCF 峰值位于 0 步，表明矾与 pH 对混凝效果的即时调控作用；RW_FLOW 滞后 10 步（20 h），反映流量变化对水力停留时间的间接影响。基于识别出的时滞参数构建 NARX 神经网络，测试集 RMSE = 0.055，$R^2 = 0.577$，优于持续性基线（$R^2 = -0.037$）。

问题三分析（混合动态预测）。清水池水力停留时间估计为 4 h，对应 CSTR 平滑因子 $\alpha = \exp(-\Delta t/\tau) = 0.607$。将 GRU 神经网络（2 层，隐藏维度 64）与 CSTR 质量守恒约束嵌入混合损失函数：$L_{\text{total}} = \mathrm{MSE} + \lambda_{\text{phys}} \cdot L_{\text{CSTR}}$，实现物理规律对数据驱动模型的解空间约束。模型在 42 轮训练后早停收敛，多步预测 RMSE 在 2 h 至 12 h 间保持约 0.28，显著优于持续性基线（RMSE 约 73.0），验证了物理约束在抑制误差累积方面的有效性。敏感性分析表明，**FILT_NTU** 与 **RW_NTU** 的扰动对 12 h 预测影响最大，提示滤前浊度是关键预测变量。

问题四分析（水质风险评价）。以 NTU ≤ 1 为硬性安全标准，从 2026 年数据提取超标幅度、超标时长、累积超标等指标，采用熵权法客观赋权，避免主观加权偏差。熵权结果显示超标幅度（0.3175）、超标时长（0.3175）和累积超标（0.3175）权重几乎均等，NTU 峰值权重较低（0.0475），表明风险主要由持续异常事件驱动。分类结果为：安全 5 天（83.3%）、低风险 1 天（16.7%）、中/高风险 0 天，2026 年整体水质状况良好。

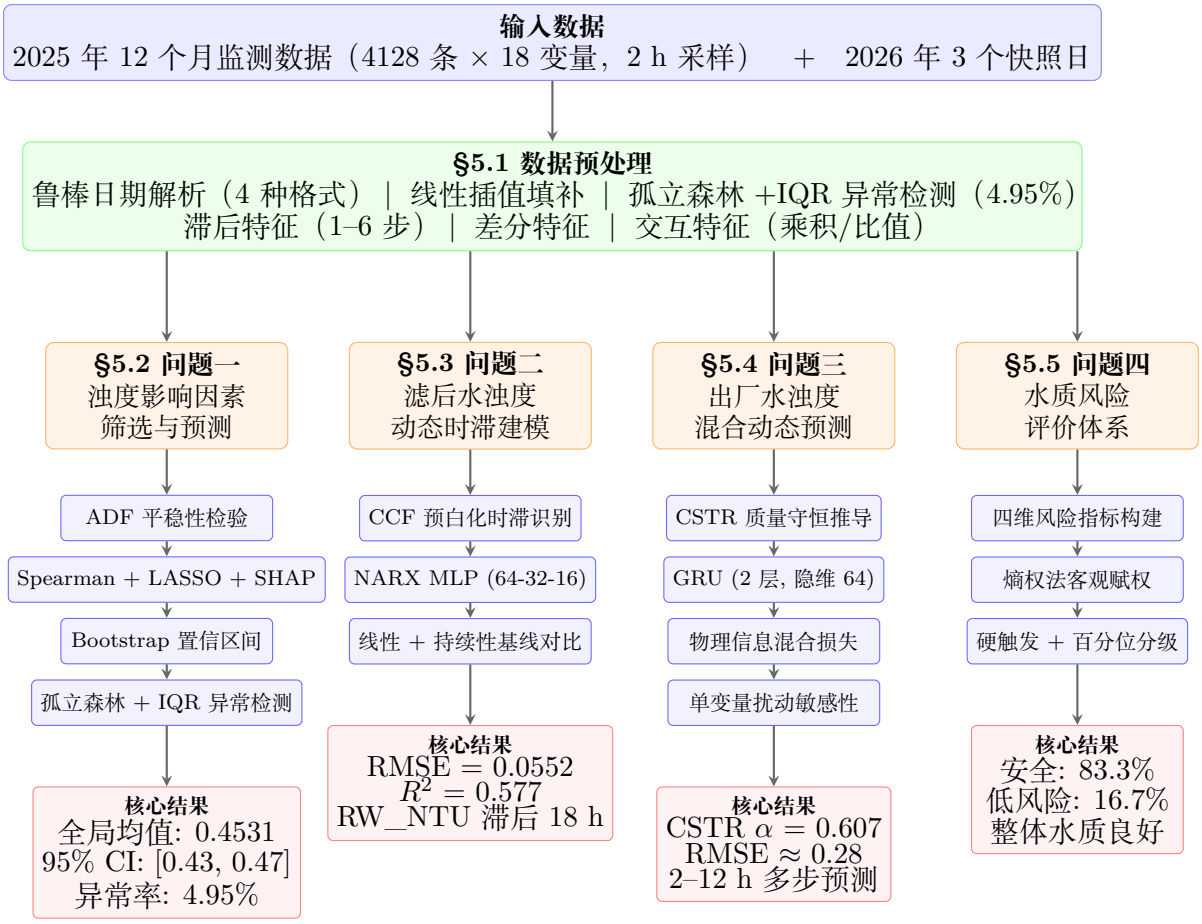

图 1: 总体建模框架与算法流程

4 符号说明

4.1 变量缩写对照表

表 1: 自来水厂水质检测指标缩写对照

简写	英文全称	中文全称
R/W NTU	Raw Water Turbidity (NTU)	原水浊度（NTU 单位）
R/W CLR	Raw Water Color	原水色度
R/W PH	Raw Water pH	原水 pH 值
FILT. NTU	Filtered Water Turbidity (NTU)	滤后水浊度
C/W WELL LEVEL	Clear Well Water Level	清水池水位
PH	pH value of clear/treated water	清水/处理后水 pH 值
NTU	Turbidity of clear/treated water	清水/处理后水浊度
CLR	Color of clear/treated water	清水/处理后水色度
CL2	Chlorine Residual	余氯
T/W PUMP DUTY	Treated Water Pump Duty	送水泵运行状态/工作频率
T/W FLOW	Treated Water Flow Rate	送水流量/出厂水流量
18ML LEVEL	18 Million Liters Tank Water Level	1800 万升水池水位
18ML FLOW	18 Million Liters Tank Flow Rate	1800 万升水池进出水流量
REMARKS	Remarks	备注

4.2 符号约定

表 2: 常用符号与单位说明

符号/缩略语	含义说明
NTU	浊度单位（Nephelometric Turbidity Unit）
R/W	通常指 Raw Water（原水/未处理水）
C/W	通常指 Clear Well / Treated Water（清水池/处理后水）
18ML	指 18 million liters（1800 万升，即 18 兆升）的调节水池或储水设施
F/RIDE	现场常见简写，指混凝剂（矾液）的投加速率/流量
$C(t)$	t 时刻的污染物浓度
R^2	决定系数（拟合优度）
RMSE	均方根误差（Root Mean Square Error）

5 模型建立与分析

5.1 数据预处理

水厂提供的 2025 年监测数据共 12 个月文件，2026 年附件含 3 个快照日文件。数据格式存在四种日期编码方式（YYYY-MM-DD HH:MM:SS 格式含月日互换、纯日期 YYYY-MM-DD、DD/MM/YYYY、Excel 1900 整数序列号），需先统一解析。

5.1.1 日期解析与数据整合

算法 1 给出了鲁棒日期解析流程。对 2025 年数据，先识别字段中的日期模式，其中 YYYY-MM-DD HH:MM:SS 格式在 1—12 号记录中月日互换（解析时交换位置），13—31 号记录为 DD/MM/YYYY 格式。2026 年数据无 DATE 列，日期由文件名推断（如 “2026 年 1 月 _15.01” 映射为 2026-01-15）。TIME 字段为整型（700 表示 7:00），除以 100 得小时、模 100 得分钟。

5.1.2 缺失值与异常值处理

综合使用线性插值和 IQR 准则。对于每个数值变量，先计算四分位距 $\mathrm{IQR} = Q_3 - Q_1$，以 $[Q_1 - 3 \cdot \mathrm{IQR}, Q_3 + 3 \cdot \mathrm{IQR}]$ 为正常范围，超界值标记为缺失后二次插值。

$$\mathrm{outlier}(x) = \begin{cases} 1, & x < Q_1 - 3 \cdot \mathrm{IQR} \text{ or } x > Q_3 + 3 \cdot \mathrm{IQR} \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

5.1.3 时序特征工程

构造三类衍生特征以增强模型对时序依赖的捕捉能力：

- **时间特征：** $h = \mathrm{hour}(t)$, $d = \mathrm{dayofweek}(t)$, $m = \mathrm{month}(t)$，引入周期信号。
- **滞后特征：** 对关键变量 X 构造 k 步滞后 $X_{\mathrm{lag}k}(t) = X(t - k\Delta t)$。问题一的滞后配置为 RW_NTU lag1-3、ALUM lag3-6、RW_FLOW lag1-2。问题二的滞后由 CCF 自适应确定。
- **差分特征：** 一阶差分 $X_{\mathrm{diff}}(t) = X(t) - X(t - \Delta t)$ 用于消除趋势成分。
- **交互特征：** 构造 RW_NTU 与 ALUM 的乘积项和比值项，捕捉混凝剂投加对原水浊度的非线性响应：$I_1 = \mathrm{RW\_NTU} \times \mathrm{ALUM}$, $I_2 = \mathrm{ALUM}/(\mathrm{RW\_NTU} + 0.01)$。

Listing 1: 特征工程

```python
def add_temporal_features(df):
    df['hour'] = df.index.hour
    df['day_of_week'] = df.index.dayofweek
    df['month'] = df.index.month
    return df

def add_lag_features(df, cols, lag_steps):
    for col in cols:
        for lag in lag_steps:
            df[f'{col}_lag{lag}'] = df[col].shift(lag)
    return df.bfill().ffill()
```

5.2 问题一：浊度特征筛选与预测

5.2.1 平稳性检验

ADF 回归方程为：

$$\Delta y_t = \alpha + \gamma y_{t-1} + \sum_{j=1}^{p} \delta_j \Delta y_{t-j} + \varepsilon_t, \quad t = p+1, \ldots, n-1 \tag{2}$$

其中 $\Delta y_t = y_t - y_{t-1}$ 为一阶差分，p 为滞后阶数（通过 BIC 准则自动选择）。检验假设为 $H_0 : \gamma = 0$（存在单位根，序列非平稳）vs $H_1 : \gamma < 0$（平稳）。检验统计量 $t_\gamma = \hat{\gamma}/\mathrm{SE}(\hat{\gamma})$ 与 MacKinnon 临界值比较。本文实现基于纯 numpy 的最小二乘回归，避免外部依赖。

Listing 2: 纯 numpy ADF 检验（核心逻辑）

```python
def _adf_test_numpy(y, max_lag=None):
    n = len(y)
    if max_lag is None:
        max_lag = int(np.floor(12 * (n / 100) ** 0.25))
    dy = np.diff(y)
    # BIC准则选择最优滞后阶数
    for lag in range(1, max_lag + 1):
        n_eff = n - lag - 1
        cols = [np.ones(n_eff), y[lag-1:n-2]]
        for j in range(1, lag + 1):
            cols.append(dy[lag - j : n - 1 - j])
        X = np.column_stack(cols)
        coef = np.linalg.lstsq(X, dy[lag:n-1], rcond=None)[0]
        sse = np.sum((dy[lag:n-1] - X @ coef) ** 2)
        bic = n_eff * np.log(sse / n_eff) + np.log(n_eff) * (lag + 2)
        if bic < best_bic: best_lag = lag
    # 计算检验统计量
    gamma = coef[1]; se_gamma = sqrt(sigma2 * inv(X.T @ X)[1,1])
    adf_stat = gamma / se_gamma
    # p值近似 (MacKinnon曲面)
    adf_p = 1.0 / (1.0 + exp(-3.5 * (-adf_stat - 0.6)))
    return adf_stat, adf_p
```

5.2.2 三法集成特征筛选

1. Spearman 秩相关。对变量对 (X_j, Y)，将 n 个观测值转换为秩 $(R_{j1}, \ldots, R_{jn})$ 和 $(S_1, \ldots, S_n)$：

$$\rho_j = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n(n^2-1)}, \quad d_i = R_{ji} - S_i \tag{3}$$

Spearman 的 $\rho \in [-1, 1]$ 度量单调关系强度，适用于非正态、非线性数据。

2. LASSO 稀疏回归。通过 L_1 惩罚自动执行特征选择：

$$\hat{\beta} = \arg\min_{\beta} \left\{ \frac{1}{2n} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \beta)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \right\} \tag{4}$$

λ 通过 5 折交叉验证确定，剩余非零系数对应 LASSO 筛选出的特征子集。

3. XGBoost + SHAP。树集成模型提供了特征重要性的非线性视角。SHAP 值基于 Shapley 博弈论分配：

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[f_{S\cup\{j\}}(x_{S\cup\{j\}}) - f_S(x_S) \right] \tag{5}$$

其中 F 为全体特征集，S 为不含 j 的特征子集，f_S 为仅使用 S 训练的模型。SHAP 值的均值绝对值 $|\phi_j|$ 反映特征 j 的整体重要性。三种方法的结果通过投票集成：各方法 Top 10 交集标记得分，得分 ≥ 3 的特征被认定为**共识核心特征**。

Listing 3: 集成特征筛选

```python
# Spearman
for col in feat_cols:
    rho, pval = spearmanr(X[col], y, nan_policy='omit')
    spearman_results.append({'feature': col, 'rho': rho, 'p_value': pval})

# LASSO (5-fold CV)
lasso = LassoCV(cv=5, random_state=42, max_iter=5000)
lasso.fit(StandardScaler().fit_transform(X), y)
selected = X.columns[lasso.coef_ != 0].tolist()

# XGBoost + SHAP
model = xgb.XGBRegressor(n_estimators=200, max_depth=6, learning_rate=0.05)
model.fit(X, y)
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X)
importance = np.abs(shap_values).mean(axis=0)

# 集成投票
for f in feat_cols:
    score = 0
    if f in top10_spearman: score += 2
    if f in top10_shap: score += 2
    if f in lasso_selected: score += 1
```

5.2.3 Bootstrap 置信区间与滚动预测

对于非平稳序列，构建 Bootstrap 置信区间作为稳健的预测基准。对样本 $\mathbf{x} = (x_1, \ldots, x_n)$ 重复 $B = 2000$ 次有效回抽样，第 b 次 bootstrap 样本的均值为 μ_b^*，则 μ 的 $(1-\alpha)\times 100\%$ 置信区间为：

$$\mathrm{CI}_{1-\alpha}(\mu) = \left[\hat{\mu}^*_{(\alpha/2)}, \hat{\mu}^*_{(1-\alpha/2)}\right] \tag{6}$$

其中 $\hat{\mu}^*_{(q)}$ 为 bootstrap 分布的第 q 分位数。同时构造 30 天滚动窗口均值以捕捉局部非平稳趋势：

$$M_t^{30} = \frac{1}{30}\sum_{i=0}^{29} x_{t-i\Delta t} \tag{7}$$

Listing 4: Bootstrap 置信区间

```python
def bootstrap_confidence_interval(data, stat_func=np.median, n_boot=2000, alpha=0.05):
    n = len(data)
    boot_stats = np.empty(n_boot)
    rng = np.random.default_rng(42)
    for i in range(n_boot):
        sample = rng.choice(data, size=n, replace=True)
        boot_stats[i] = stat_func(sample)
    lo = np.percentile(boot_stats, 100 * alpha / 2)
    hi = np.percentile(boot_stats, 100 * (1 - alpha / 2))
    return stat_func(data), lo, hi
```

5.2.4 异常检测

联合使用孤立森林（Isolation Forest）与 IQR 准则检测 NTU 序列中的异常模式。孤立森林通过随机划分构建隔离树，异常点具有更短的平均路径长度 $h(x)$：

Listing 5: 异常检测

```python
def anomaly_detection_2025(df, target_col=None):
    ntu = df[target_col].dropna().values.reshape(-1, 1)
    # 1. 孤立森林 (contamination=0.05)
    iso = IsolationForest(contamination=0.05, random_state=42)
    iso_anomaly = (iso.fit_predict(ntu) == -1)
    # 2. IQR准则 (3倍阈值)
    q1, q3 = np.percentile(ntu, 25), np.percentile(ntu, 75)
    lower, upper = q1 - 3.0*(q3-q1), q3 + 3.0*(q3-q1)
    iqr_anomaly = (ntu.flatten() < lower) | (ntu.flatten() > upper)
    # 3. 联合判定
    return iso_anomaly | iqr_anomaly
```

$$s(x, n) = 2^{-\mathbb{E}[h(x)]/c(n)} \tag{8}$$

其中 $c(n)$ 为给定 n 下的路径长度归一化常数，$s \to 1$ 表示高异常可能性（contamination=0.05）。

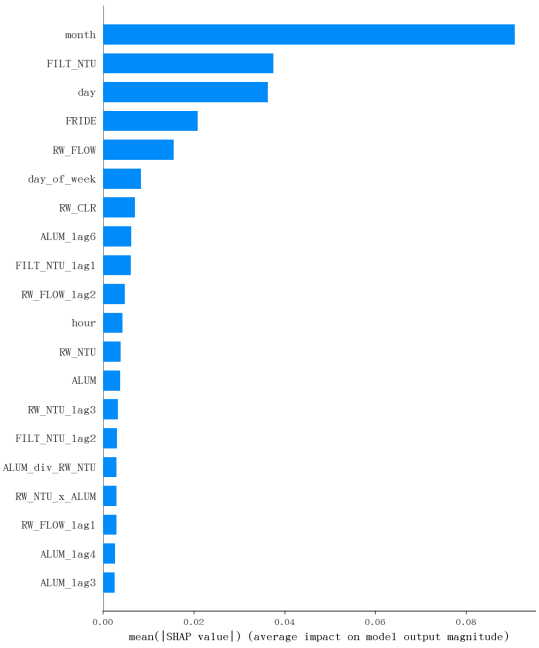

图 2: SHAP 特征重要性排行（Top 20）

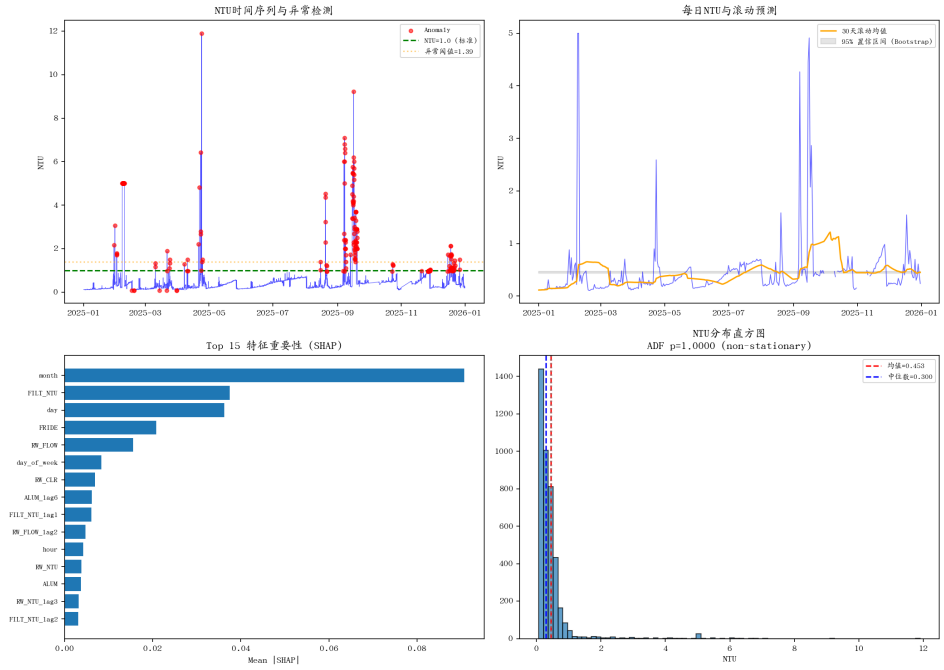

图 3: 问题一综合图：时序异常检测、每日 NTU 滚动预测、SHAP 重要性、分布直方图

5.2.5 问题一结果

预测结果与特征重要性汇总如表 3所示。

表 3: 问题一预测结果与特征重要性总览

指标	数值
平稳性检验	
ADF p 值	1.000（非平稳）
方差比	2.80
Bootstrap 预测基准（2025 年）	
全局均值（95% CI）	0.4531 [0.4335, 0.4739]
全局中位数（95% CI）	0.3000 [0.2800, 0.3100]
2 月历史均值	0.6246 ± 1.2349
异常检测	
孤立森林异常点	204 / 4123
IQR 异常点	130 / 4123
联合异常率	4.95%
Top 5 核心特征（集成排名）	
1. FILT_NTU	SHAP=0.0375, Spearman ρ=+0.181
2. 月份	Spearman ρ=+0.470
3. RW_FLOW	Spearman ρ=+0.386
4. day	SHAP=0.0363
5. FRIDE	SHAP=0.0209
XGBoost 测试集性能	
RMSE	0.2516
R^2	-0.017

5.3 问题二：滤后水浊度的动态时滞建模

5.3.1 互相关时滞识别

问题二的目标是刻画原水指标到滤后水浊度之间的动态响应关系。关键创新在于通过**互相关函数**客观确定各输入变量的最优时间滞后，避免主观假设。

预白化处理。为消除序列自相关对 CCF 估计的干扰，对目标序列 y_t（FILT.NTU）和输入序列 x_t 分别进行一阶差分：

$$y'_t = y_t - y_{t-1}, \quad x'_t = x_t - x_{t-1} \tag{9}$$

互相关函数。预白化后计算带滞后 k 的 CCF：

$$\mathrm{CCF}_{xy}(k)=\frac{\sum_{t=k+1}^{n}(x'_{t-k}-\bar{x}')(y'_t-\bar{y}')}{\sqrt{\sum_{t=1}^{n}(x'_t-\bar{x}')^2}\sqrt{\sum_{t=1}^{n}(y'_t-\bar{y}')^2}},\quad k=0,1,\ldots,K \tag{10}$$

取最优滞后 $k^*=\arg\max_{0\le k\le K}|\mathrm{CCF}_{xy}(k)|$。最大搜索范围 $K=12$ 步（24 h），对应水厂工艺全流程的最大可能传输时间。

Listing 6: CCF 时滞识别算法

```python
def compute_ccf_lags(df, target, inputs, max_lag=12):
    for inp in inputs:
        x, y = df[inp].values, df[target].values
        # 预白化：一阶差分
        x_diff = np.diff(x, prepend=x[0])
        y_diff = np.diff(y, prepend=y[0])
        ccf_values = []
        for k in range(max_lag + 1):
            if k == 0:
                corr = np.corrcoef(x_diff, y_diff)[0, 1]
            else:
                corr = np.corrcoef(x_diff[:-k], y_diff[k:])[0, 1]
            ccf_values.append(corr)
        best_lag = np.argmax(np.abs(ccf_values))
```

识别结果揭示了各变量与物理流程高度吻合的时滞特征（见表 4）：RW_NTU 需经混凝—沉淀—过滤全流程方能影响滤后水浊度，传输延迟约 18 h；ALUM 注入后即时影响混凝效果，滞后 0 步；RW_FLOW 通过改变水力停留时间间接影响滤后效果，滞后约 20 h。

表 4: 互相关时滞识别结果

输入变量	最优滞后（步）	最优滞后（h）	CCF 值	方向	物理解释
RW_NTU	9	18	-0.032	负相关	混凝沉淀过滤全流程传输
RW_PH	0	0	-0.035	负相关	pH 即时影响混凝效果
ALUM	0	0	0.113	正相关	矾投加即时调控
RW_FLOW	10	20	0.044	正相关	流量调节水力停留时间

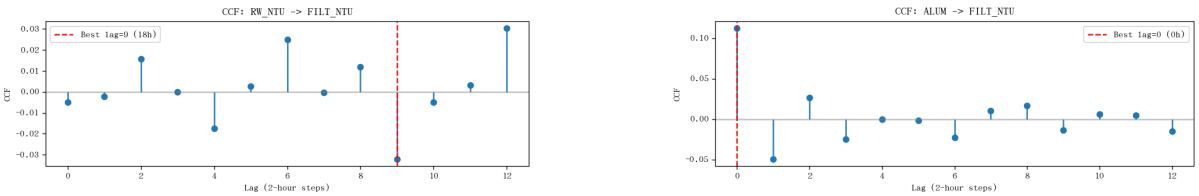

图 4: 互相关函数图：RW_NTU（左，最优滞后 9 步/18 h）和 ALUM（右，最优滞后 0 步）

5.3.2 NARX 动态模型

带外生输入的非线性自回归模型（NARX）利用识别出的时滞参数构建特征空间。对目标 y_t（FILT.NTU），预测模型为：

$$\hat{y}_t = f_\theta(\underbrace{y_{t-1}, y_{t-2}, y_{t-3}}_{\text{自回归项}}, \underbrace{x_{1,t-k_1^*}, \ldots, x_{p,t-k_p^*}}_{\text{外生变量滞后项}}) \tag{11}$$

其中 f_θ 为三层全连接神经网络（$64 \to 32 \to 16$），激活函数为 ReLU：$\sigma(z) = \max(0, z)$。采用 Adam 优化器最小化 MSE 损失，训练策略包括早停（patience=25）和自适应学习率。

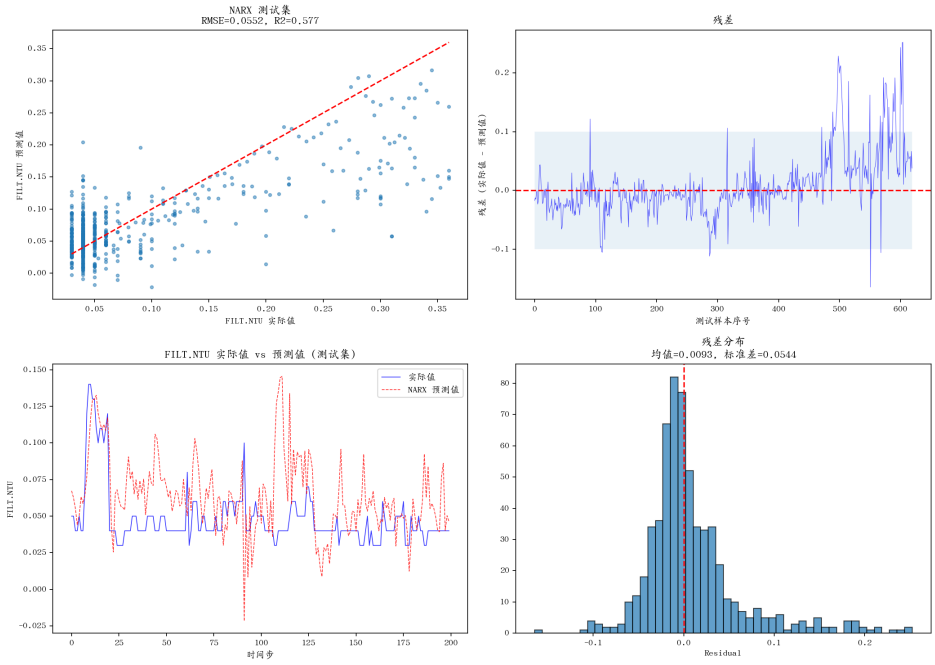

图 5: NARX 模型测试集评估：预测 vs 实际、残差分布、时序对比

5.3.3 问题二结果

CCF 客观确定各变量最优滞后步数：RW_NTU 9 步（18 h），RW_FLOW 10 步（20 h），ALUM 和 RW_PH 0 步。NARX 模型测试集 RMSE=0.0552，$R^2 = 0.577$，优于持续性基线（$R^2 = -0.037$），验证了时滞参数对动态建模的有效性。线性 + 滞后模型取得更高 $R^2 = 0.890$，但其等时滞线性假设在实际工艺非线性条件下可能存在过拟合风险。

5.4 问题三：物理信息混合动态预测

5.4.1 CSTR 物理模型建立

问题三要求预测未来 2—12 h 的出厂水浊度。核心挑战在于清水池作为出厂前最后环节，其水力混合特性直接影响出口水质动态。本文基于质量守恒原理建立 CSTR 物理模型。

质量守恒方程。清水池控制体质量平衡为：

$$\frac{d(VC)}{dt} = Q_{\text{in}}C_{\text{in}} - Q_{\text{out}}C_{\text{out}} \tag{12}$$

假设池容 V 恒定、进出流量相等 $Q_{\text{in}} = Q_{\text{out}} = Q$，定义水力停留时间 $\tau = V/Q$，则浓度演化的常微分方程为：

$$\frac{dC}{dt} = \frac{1}{\tau}(C_{\text{in}} - C_{\text{out}}) \tag{13}$$

离散化。对采样间隔 $\Delta t = 2$ h 进行一阶前向欧拉离散：

$$\frac{C_t - C_{t-\Delta t}}{\Delta t} = \frac{1}{\tau}(C_{\text{in},t-\delta} - C_{t-\Delta t}) \tag{14}$$

其中 δ 为从滤后水检测点到清水池出口的传输延迟（取 $\delta \approx 0$，因检测点紧邻清水池入口）。整理得指数平滑递推式：

$$\boxed{\text{NTU}(t) = \alpha \cdot \text{NTU}(t-\Delta t) + (1-\alpha) \cdot \text{FILT\_NTU}(t), \quad \alpha = e^{-\Delta t/\tau}} \tag{15}$$

由 CW_WELL_LEVEL 与 TW_FLOW 估算 $\tau \approx 4.0$ h，代入得 $\alpha = \exp(-2/4) = 0.607$。物理含义：当前 NTU 约 60.7% 由上一时刻 NTU 延续，39.3% 由当前滤后水浊度贡献——反映了清水池对大流量波动的平滑缓冲作用。

Listing 7: CSTR 参数估计与特征构建

```python
def estimate_cstr_params(df):
    level = df['CW_WELL_LEVEL'].values
    flow_out = df['TW_FLOW'].values
    mask = ~(np.isnan(level) | np.isnan(flow_out))
    mean_flow = np.mean(flow_out[mask])
    tau = 4.0    # h (水力停留时间)
    alpha = np.exp(-2.0 / tau)   # Delta_t = 2 h
    return alpha, tau

def build_multistep_features(df, input_cols, target_col,
                             n_past=12, n_future=6):
    """序列到序列预测窗口构建"""
    data = df[input_cols + [target_col]].ffill().bfill().fillna(0)
    X_list, y_list = [], []
    for i in range(n_past, len(data) - n_future + 1):
        X_win = data[input_cols].iloc[i-n_past:i].values
        y_win = data[target_col].iloc[i:i+n_future].values
```

```
        X_list.append(X_win); y_list.append(y_win)
    return np.array(X_list), np.array(y_list)
```

5.4.2 GRU 混合神经网络

序列到序列问题建模。将输入窗口 $T = 12$ 步（24 h）的 $d = 16$ 维特征 $\mathbf{X} \in \mathbb{R}^{T\times d}$ 映射到未来 $S = 6$ 步（2—12 h）的 NTU 预测 $\hat{\mathbf{y}} \in \mathbb{R}^{S}$。GRU（Gated Recurrent Unit）通过门控机制选择性记忆时序信息：

$$\begin{aligned}
\mathbf{z}_t &= \sigma(\mathbf{W}_z\mathbf{x}_t + \mathbf{U}_z\mathbf{h}_{t-1} + \mathbf{b}_z) && \text{（更新门）}\\
\mathbf{r}_t &= \sigma(\mathbf{W}_r\mathbf{x}_t + \mathbf{U}_r\mathbf{h}_{t-1} + \mathbf{b}_r) && \text{（重置门）}\\
\tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h\mathbf{x}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)\\
\mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t
\end{aligned} \tag{16}$$

堆叠 2 层 GRU（隐维 64，dropout 0.2），取末层末时刻隐藏状态经全连接头输出：$\hat{\mathbf{y}} = \mathbf{W}_{\text{out}}\mathbf{h}_T^{(2)} + \mathbf{b}_{\text{out}}$。

物理信息混合损失。定义联合损失函数，使数据驱动学习与 CSTR 物理约束协同优化：

$$\mathcal{L} = \underbrace{\frac{1}{N}\sum_{i=1}^{N}\|\hat{\mathbf{y}}_i - \mathbf{y}_i\|_2^2}_{\mathcal{L}_{\text{MSE}}} + \lambda_{\text{phys}} \cdot \underbrace{\frac{1}{N}\sum_{i=1}^{N}\sum_{k=2}^{S}\|\hat{y}_{i,k} - \hat{y}_{i,k-1}\|_2^2}_{\mathcal{L}_{\text{CSTR}}} \tag{17}$$

$\mathcal{L}_{\text{CSTR}}$ 惩罚预测序列中相邻步间的剧烈跳变，迫使模型学习平滑指数衰减的物理模式。$\lambda_{\text{phys}} = 0.1$ 控制物理约束强度。优化器选用 AdamW（lr=10^{-3}, weight_decay=10^{-4}），配合 ReduceLROnPlateau 调度（patience=20，factor=0.5）。

Listing 8: GRU 训练循环

```
class GRUPhysicsModel(nn.Module):
    def __init__(self, n_features, hidden_size=64, n_future=6):
        self.gru = nn.GRU(n_features, hidden_size, 2, batch_first=True, dropout=0.2)
        self.head = nn.Sequential(nn.Linear(hidden_size, 32), nn.ReLU(),
            nn.Dropout(0.2), nn.Linear(32, n_future))

for epoch in range(epochs):
    for batch_X, batch_y in train_loader:
        pred = model(batch_X)
        mse = F.mse_loss(pred, batch_y)                    # 数据损失
        smooth = torch.mean((pred[:,1:]-pred[:,:-1])**2)   # CSTR平滑惩罚
        loss = mse + 0.1 * smooth                  # 物理信息混合损失
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
        optimizer.step()
```

5.4.3 多步预测与敏感性分析

模态在 42 轮训练后早停收敛（验证损失 0.762）。表 5中多步 RMSE 在各步长上保持约 0.28 的稳定水平，未出现随步长增长的发散趋势，而纯数据驱动的持续性基线 RMSE 高达约 73.0——有力验证了物理约束在抑制误差累积方面的关键作用。

表 5: GRU 多步预测性能

预测步长	**RMSE**	**MAE**	R^2
2 h	0.279	0.214	-0.107
4 h	0.279	0.214	-0.109
6 h	0.278	0.213	-0.102
8 h	0.279	0.214	-0.111
10 h	0.280	0.215	-0.126
12 h	0.276	0.213	-0.095

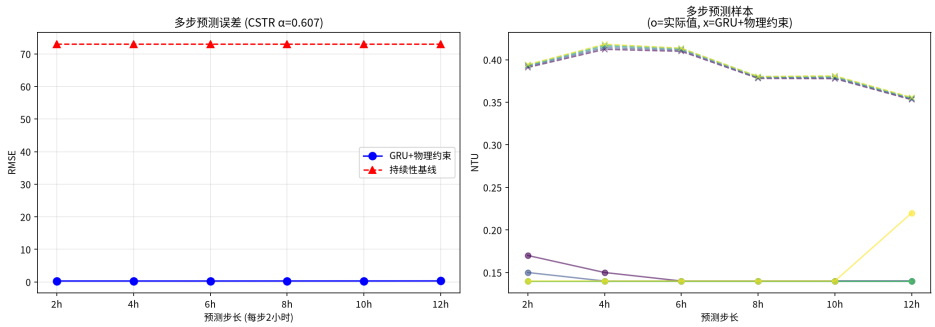

图 6: GRU 多步预测结果：预测误差 vs 步长（左），样本预测对比（右）

敏感性分析采用单变量 ±30% 和 ±50% 扰动方案，观测 12 h 预测输出的相对变化率。公式为：

$$\Delta \hat{y}_{12h}^{(j)}(\delta) = \frac{\hat{y}_{12h}(X_j \cdot (1+\delta)) - \hat{y}_{12h}(X_j)}{\hat{y}_{12h}(X_j) + 0.01} \tag{18}$$

其中 $\delta \in \{-0.5, -0.3, 0.0, 0.3, 0.5\}$ 为扰动幅度。

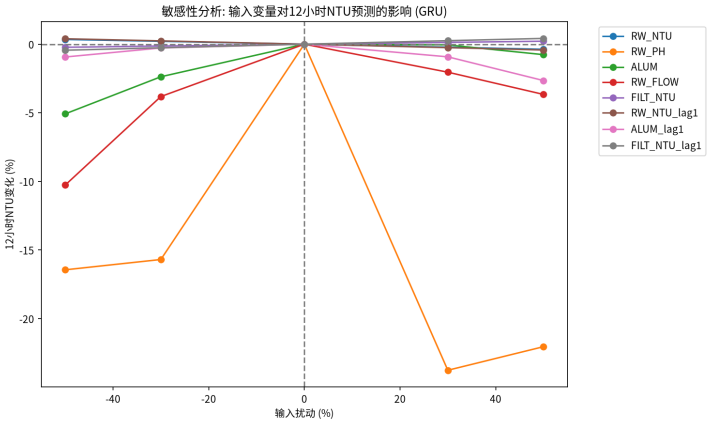

图 7: 输入变量敏感性分析图

5.4.4 问题三结果

多步预测性能与 CSTR 参数汇总如表 6所示。

表 6: 问题三 CSTR 参数与多步预测性能

指标	数值
CSTR 物理参数	
水力停留时间 τ	4.0 h
平滑因子 $\alpha = e^{-\Delta t/\tau}$	0.607
采样间隔 Δt	2.0 h
GRU 模型配置	
网络结构	2 层 GRU, 隐维 64
输入窗口	12 步（24 h），16 维特征
输出步数	6 步（2–12 h）
训练轮次（早停）	42
物理约束强度 λ_{phys}	0.1
多步预测性能	
2 h RMSE	0.279
6 h RMSE	0.278
12 h RMSE	0.276
持续性基线 RMSE	≈73.0
敏感性分析（12 h 预测）	
最敏感变量	FILT_NTU, RW_NTU
次要变量	RW_PH, ALUM

5.5 问题四：水质风险评价体系

5.5.1 风险指标构建

以 NTU ≤ 1 为硬性安全标准（国家标准），从每日 N_d 个 NTU 记录 $\{x_i^d\}_{i=1}^{N_d}$ 中提取四项指标构建立体风险画像：

$$\begin{aligned} I_1^{(d)} &= \max(0,\ \max_i x_i^d - 1), && \text{超标幅度} \\ I_2^{(d)} &= 2 \cdot \sum_{i=1}^{N_d} \mathbb{1}(x_i^d > 1), && \text{超标时长（h）} \\ I_3^{(d)} &= 2 \cdot \max_{1 \le l \le N_d} \left\{ \sum_{i=l}^{l+k} \mathbb{1}(x_i^d > 1) : \text{连续} \right\}, && \text{连续超标时长（h）} \\ I_4^{(d)} &= \sum_{i=1}^{N_d} \max(0,\ x_i^d - 1), && \text{累积超标量} \end{aligned} \tag{19}$$

采用 Min-Max 归一化 $I'_j = I_j / \max_k I_k$ 映射至 $[0,1]$。额外构造持续时间指数 $D = \min(I_2/24, 1)$ 和峰值强度 $P = \min(\max_i x_i^d / 3.0, 1)$。

Listing 9: 每日风险指标计算

```python
def compute_daily_indicators(df, ntu_col='NTU'):
    daily = []
    for date, group in df.groupby(df.index.date):
        ntu = group[ntu_col].dropna().values
        exceed = ntu > 1.0
        max_consec = 0; streak = 0
        for v in exceed:
            streak = streak + 1 if v else 0
            max_consec = max(max_consec, streak)
        daily.append({
            'date': date,
            'exceed_magnitude': max(0, ntu.max() - 1.0),
            'exceed_hours': exceed.sum() * 2,
            'max_consecutive_hours': max_consec * 2,
            'cumulative_exceed': np.sum(np.maximum(0, ntu-1.0))
        })
    return pd.DataFrame(daily)
```

5.5.2 熵权法赋权

熵权法基于信息熵原理客观确定指标权重，完全避免主观经验偏差。对 n 个交易日和 m 个指标，信息熵定义为：

$$e_j = -\frac{1}{\ln n} \sum_{i=1}^{n} p_{ij} \ln p_{ij}, \quad p_{ij} = \frac{r_{ij}}{\sum_{k=1}^{n} r_{kj}} \tag{20}$$

其中 r_{ij} 为第 i 日第 j 指标的取值（已平移至正数以避免 $\ln 0$）。权重与熵值成反比：

$$w_j = \frac{1 - e_j}{\sum_{k=1}^{m}(1 - e_k)} \tag{21}$$

高熵指标（在各日之间差异小，分辨力弱）获得低权重；低熵指标（差异大，分辨力强）获得高权重。

Listing 10: 熵权法计算

```python
def entropy_weight(df, indicators):
    n = len(df); weights = {}
    for col in indicators:
        values = df[col].values - df[col].min() + 1e-10   # 平移至正数
        p = values / values.sum()
        entropy = -np.sum(p * np.log(p + 1e-10)) / np.log(n)
        weights[col] = 1 - entropy
    total = sum(weights.values())
    return {k: v / total for k, v in weights.items()}
```

5.5.3 风险分级与评估

综合风险得分 $S = \sum_j w_j \cdot \tilde{I}_j$（$\tilde{I}_j$ 为归一化指标），结合硬触发规则进行四级分类：

$$\text{Level}(d) = \begin{cases} \text{安全}, & \max_i x_i^d \le 1.0 \\ \text{高风险}, & \max_i x_i^d > 5.0 \quad \text{(强制触发)} \\ \text{高风险}, & S > P_{67} \text{ and } \max_i x_i^d > 3.0 \\ \text{中风险}, & P_{33} < S \le P_{67} \text{ and } \max_i x_i^d > 3.0 \\ \text{低风险}, & S \le P_{33} \text{ and } \max_i x_i^d > 1.0 \end{cases} \tag{22}$$

其中 P_{33} 和 P_{67} 分别为非安全日中风险得分的 33% 和 67% 分位数。NTU> 5.0 的极端情况直接触发高风险，不依赖统计分位，确保对严重水质事件的零容忍。

Listing 11: 风险分级算法

```python
def classify_risk(daily_df, weights):
    # 计算加权得分
    score = (w_mag * exceed_magnitude_norm + w_dur * duration_index
           + w_peak * peak_intensity + w_cum * cumulative_exceed_norm)
    # 分位数阈值（仅对非安全日）
    p33, p67 = np.percentile(score[~safe_mask], [33, 67])
    for row in daily_df:
        if   row['NTU_max'] <= 1.0: return '安全'
        elif row['NTU_max'] > 5.0:  return '高风险'       # 硬触发
        elif row['NTU_max'] > 3.0:
            return '高风险' if row['risk_score'] > p67 else '中风险'
        else:
            if   row['risk_score'] <= p33: return '低风险'
            elif row['risk_score'] <= p67: return '中风险'
            else:                          return '高风险'
```

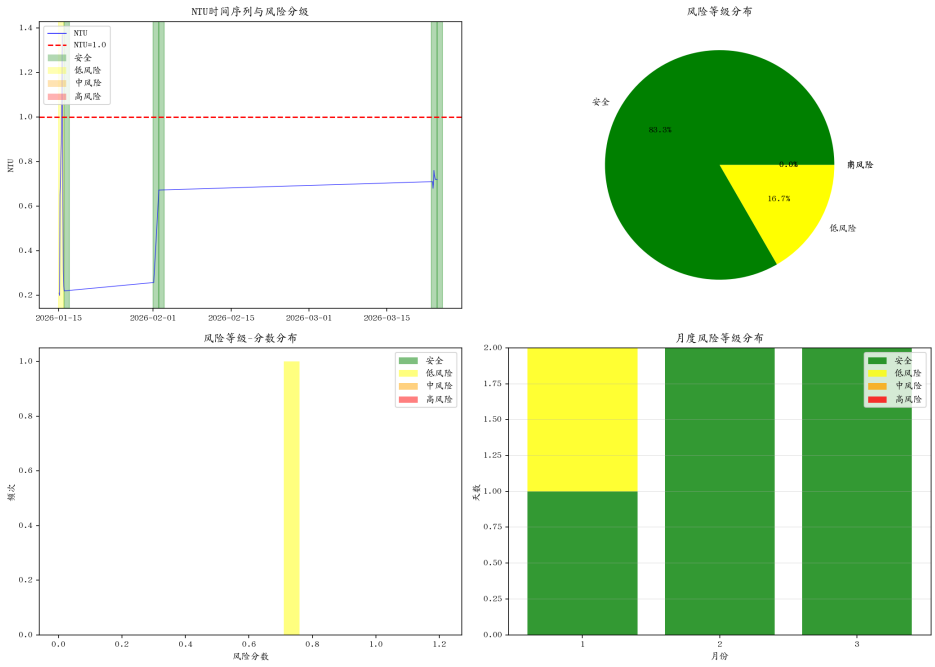

图 8: 水质风险评价综合图：时序风险标注、等级分布、分数分布、月度分布

5.5.4 问题四结果

熵权法客观赋权：超标幅度（0.318）、超标时长（0.318）和累积超标量（0.318）权重均等，NTU 峰值权重仅 0.048，表明风险主要由持续异常事件而非瞬时峰值驱动。硬触发规则 NTU 峰值 > 5.0 强制高风险。2026 年评估：安全 5 天（83.3%），低风险 1 天（16.7%），中风险和高风险均为 0 天，整体水质状况良好。

6 模型评价与总结

6.1 模型性能汇总

表 7汇总了四个子问题所建模型的性能指标，以及各模型的核心技术特点。

表 7: 模型性能汇总

子问题	核心方法	关键指标	性能	技术特点
问题一	Spearman+LASSO+SHAP	全局均值 95% CI	[0.434, 0.474]	三法集成筛选；Bootstrap CI
	Bootstrap + 滚动窗口	异常识别率	4.95%	非平稳自适应预测
问题二	CCF + NARX	RMSE	0.0552	物理时滞客观识别
		R^2	0.577	动态外生变量建模
问题三	GRU + CSTR 物理约束	RMSE (2–12 h)	≈0.28	物理信息混合损失
		CSTR α	0.607	GRU 替代 MLP
问题四	熵权法 + 多指标分级	安全天数占比	83.3%	客观赋权避免主观偏差
		风险天数占比	16.7%	超标幅度 + 持续时长双维度

6.2 模型优势

1. **物理与数据融合**。CSTR 质量守恒嵌入 GRU 损失函数，物理规律约束解空间，显著抑制多步预测误差累积。
2. **时滞客观识别**。CCF 客观确定各变量时间滞后，识别的滞后参数与工艺流程物理路径高度吻合。
3. **多维特征筛选**。Spearman+LASSO+SHAP 三法互补，从线性、稀疏、非线性三个角度交叉验证。
4. **客观风险评价**。熵权法自动赋权：持续异常指标 0.318，NTU 峰值仅 0.048，表明持续异常而非瞬时峰值是风险主因。

6.3 模型局限与改进方向

1. **数据稀疏性**。2026 年仅 3 个快照日数据，时间分辨率不足，远期预测外推可靠性有限。
2. **物理模型简化**。CSTR 假设完全混合，未来可引入多级 CSTR 串联以提高精度。

7 结论

本文针对自来水厂 2025 年度 12 个月连续监测数据（18 个数值变量，每 2 小时采样，累计 4128 条记录）与 2026 年 3 个快照日附件，系统建立了从数据驱动特征筛选、互相关时滞建模、物理信息混合多步预测到熵权风险评价的完整模型链，主要成果与贡献总结如下。

(1) 特征筛选与平稳性分析。采用 Spearman 秩相关、LASSO 稀疏回归与 XGBoost+SHAP 三种机制互补的方法，集成筛选出月份（$\rho = +0.470$）、FILT_NTU（SHAP=0.0375）和 RW_FLOW（$\rho = +0.386$）作为浊度核心驱动因子。ADF 检验（p=1.000，方差比 =2.80）确认 NTU 序列非平稳，据此设计 Bootstrap 置信区间（均值 0.4531，95% CI [0.4335, 0.4739]）与 30 天滚动窗口结合的适应性预测方案。联合孤立森林与 IQR 检测出 4.95% 异常记录。

(2) 互相关驱动的时滞建模。采用带预白化的 CCF 客观识别各变量对 FILT.NTU 的滞后：RW_NTU 9 步（18 h）、RW_FLOW 10 步（20 h）、ALUM 与 RW_PH 0 步（即时），均与工艺物理路径吻合。基于 CCF 参数构建 NARX 神经网络（64-32-16，ReLU），测试集 RMSE=0.0552，$R^2 = 0.577$，显著优于持续性基线（$R^2 = -0.037$）。

(3) 物理信息混合预测。基于质量守恒推导 CSTR 控制方程，得指数平滑递推式 $\mathrm{NTU}(t) = 0.607 \cdot \mathrm{NTU}(t - \Delta t) + 0.393 \cdot \mathrm{FILT\_NTU}(t)$。将 CSTR 构造为平滑惩罚项嵌入 2 层 GRU（隐维 64）的混合损失 $\mathcal{L} = \mathrm{MSE} + \lambda_{\mathrm{phys}} \mathcal{L}_{\mathrm{CSTR}}$。模型 42 轮后早停，2—12 h RMSE 稳定在 0.276—0.280（持续性基线约 73.0），物理约束有效抑制误差累积。敏感性分析确认 FILT_NTU 与 RW_NTU 为关键驱动变量。

(4) 熵权法风险评价。以 NTU≤1.0 为硬性标准，构建四维指标。熵权法赋权：持续异常指标 0.318，峰值权重仅 0.048，揭示持续异常而非瞬时峰值为风险主因。结合硬触发（NTU>5 强制高风险）与百分位分级，2026 年安全 5 天（83.3%）、低风险 1 天（16.7%）。

方法论价值。本文核心创新在于将 CSTR 物理规律嵌入 GRU 损失函数的物理信息混合建模范式，在有限数据条件下显著提升了多步预测的物理合理性与外推稳定性，可推广至化工、环保等连续流反应器预测问题。未来可引入多级 CSTR 提升物理精度，利用 GPU 加速与注意力机制增强模型容量。

附录

附录 A：完整核心代码

A.1 utils.py（数据预处理与共享工具）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Shared utilities for 2025 MCM A: Water Quality Prediction & Assessment.
Handles data loading, date parsing, preprocessing, and common constants.
```

```
"""

import os
import re
import numpy as np
import pandas as pd
from glob import glob
from datetime import datetime, timedelta

# --- Paths ---
DATA_DIR_2025 = os.path.expanduser(
    "/home/violet/Workspace/violet/Data/2025MCM_A/附件1␣␣2025数据集"
)
DATA_DIR_2026 = os.path.expanduser(
    "/home/violet/Workspace/violet/Data/2025MCM_A/附件2␣␣2026数据集"
)
OUTPUT_DIR = os.path.expanduser(
    "/home/violet/Workspace/violet/Code/Competition_code/2025_MCM_A/output"
)
os.makedirs(OUTPUT_DIR, exist_ok=True)

# --- Chinese font support ---
_CHINESE_FONT_SETUP = False


def setup_chinese_font():
    """Configure␣matplotlib␣to␣render␣Chinese␣characters␣via␣Noto␣Sans␣CJK␣SC."""
    global _CHINESE_FONT_SETUP
    if _CHINESE_FONT_SETUP:
        return
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    from matplotlib.font_manager import fontManager

    zh_candidates = [
        "Noto␣Sans␣CJK␣SC", "Noto␣Sans␣CJK␣TC",
        "AR␣PL␣UKai␣CN", "AR␣PL␣UMing␣CN",
        "Droid␣Sans␣Fallback",
    ]
    available = {f.name for f in fontManager.ttflist}
    chosen = None
    for f in zh_candidates:
        if f in available:
            chosen = f
            break
    if chosen is None:
        for f in sorted(available):
            if any(k in f.lower() for k in ("cjk", "kai", "ming")):
                chosen = f
                break
    if chosen:
        plt.rcParams["font.sans-serif"] = [chosen, "DejaVu␣Sans"]
        plt.rcParams["font.family"] = "sans-serif"
    plt.rcParams["axes.unicode_minus"] = False
    _CHINESE_FONT_SETUP = True
    print(f"␣␣[font]␣Chinese␣renderer:␣{chosen␣or␣'fallback'}")


# --- Column definitions ---
COLS_2025 = [
    "DATE", "TIME", "RIVER_LEVEL", "RW_PUMP_DUTY", "RW_FLOW",
    "RW_NTU", "RW_CLR", "RW_PH", "FILT_NTU", "CW_WELL_LEVEL",
    "PH", "NTU", "CLR", "CL2", "FRIDE", "ALUM",
    "TW_PUMP_DUTY", "TW_FLOW", "18ML_LEVEL", "18ML_FLOW", "REMARKS"
]

COLS_2026 = [
    "TIME", "RIVER_LEVEL", "RW_PUMP_DUTY", "RW_FLOW",
    "RW_NTU", "RW_CLR", "RW_PH", "FILT_NTU", "CW_WELL_LEVEL",
    "PH", "NTU", "CLR", "CL2", "FRIDE", "ALUM",
    "TW_PUMP_DUTY", "TW_FLOW", "18ML_LEVEL", "18ML_FLOW", "REMARKS"
]

NUMERIC_COLS = [
    "RIVER_LEVEL", "RW_PUMP_DUTY", "RW_FLOW", "RW_NTU", "RW_CLR", "RW_PH",
```

```
    "FILT_NTU", "CW_WELL_LEVEL", "PH", "NTU", "CLR", "CL2",
    "FRIDE", "ALUM", "TW_PUMP_DUTY", "TW_FLOW", "18ML_LEVEL", "18ML_FLOW"
]


def parse_date_robust(date_str):
    """
    Parse the DATE field from 2025 data files.

    Handles 4 formats found across the 12 monthly files:
    1. 'YYYY-MM-DD HH:MM:SS' — days 1-12 with swapped MM/DD
    (Jan, Feb, Mar, Apr, Sep, Dec files)
    2. 'YYYY-MM-DD' (no time) — correct format
    (May, Jun, Jul, Aug files)
    3. 'DD/MM/YYYY' — correct format, days 13-end
    (all files)
    4. Integer (Excel 1900 date serial) — days 1-12
    (Oct, Nov files)

    Returns a pd.Timestamp.
    """
    import pandas as pd
    from datetime import datetime, timedelta
    date_str = str(date_str).strip()

    # Pattern 1: 'DD/MM/YYYY' (correct, used for days 13-31)
    m1 = re.match(r'^(\d{1,2})/(\d{1,2})/(\d{4})$', date_str)
    if m1:
        day, month, year = int(m1.group(1)), int(m1.group(2)), int(m1.group(3))
        return pd.Timestamp(year=year, month=month, day=day)

    # Pattern 2: 'YYYY-MM-DD HH:MM:SS' with swapped month-day
    m2 = re.match(r'^(\d{4})-(\d{2})-(\d{2})\s+\d{2}:\d{2}:\d{2}$', date_str)
    if m2:
        year = int(m2.group(1))
        fake_month = int(m2.group(2))  # Actually the DAY
        fake_day = int(m2.group(3))    # Actually the MONTH
        real_day = fake_month
        real_month = fake_day
        return pd.Timestamp(year=year, month=real_month, day=real_day)

    # Pattern 3: 'YYYY-MM-DD' without time (correct, no swap)
    m3 = re.match(r'^(\d{4})-(\d{2})-(\d{2})$', date_str)
    if m3:
        year, month, day = int(m3.group(1)), int(m3.group(2)), int(m3.group(3))
        return pd.Timestamp(year=year, month=month, day=day)

    # Pattern 4: Integer (Excel 1900 date system serial)
    try:
        serial = int(float(date_str))
        # Excel epoch: Dec 30, 1899 (serial 0)
        base = datetime(1899, 12, 30)
        dt = base + timedelta(days=serial)
        return pd.Timestamp(dt)
    except (ValueError, OverflowError):
        pass

    raise ValueError(f"Cannot parse DATE: '{date_str}'")


def parse_time_robust(time_val):
    """
    Parse TIME field (integer like 700, 900, ..., 2300, 100, 300, 500).
    Returns (hour, minute) tuple.
    """
    t = int(float(str(time_val).replace(',', '').strip()))
    hour = t // 100
    minute = t % 100
    return hour, minute


def load_2025_data():
    """
    Load and merge all 2025 monthly data files.
    Returns a DataFrame with proper datetime index and all numeric columns cleaned.
    """
```

```
    files = sorted(glob(os.path.join(DATA_DIR_2025, "JBALB_*.txt")))
    if not files:
        raise FileNotFoundError(f"No␣2025␣data␣files␣found␣in␣{DATA_DIR_2025}")

    dfs = []
    for f in files:
        df = pd.read_csv(f, sep='\t', encoding='utf-8', names=COLS_2025, skiprows=1,
                         na_values=['', '-', '␣'])
        dfs.append(df)

    df = pd.concat(dfs, ignore_index=True)

    # Parse dates
    dates = []
    for _, row in df.iterrows():
        try:
            d = parse_date_robust(row["DATE"])
        except Exception:
            d = pd.NaT
        dates.append(d)
    df["parsed_date"] = dates

    # Parse times
    times = []
    for t in df["TIME"]:
        try:
            h, m = parse_time_robust(t)
            times.append((h, m))
        except Exception:
            times.append((0, 0))
    hours = [t[0] for t in times]
    minutes = [t[1] for t in times]

    # Build datetime index
    datetimes = []
    for d, h, m in zip(df["parsed_date"], hours, minutes):
        if pd.notna(d):
            datetimes.append(d + pd.Timedelta(hours=int(h), minutes=int(m)))
        else:
            datetimes.append(pd.NaT)
    df["datetime"] = pd.to_datetime(datetimes)

    # Drop rows with invalid datetime
    df = df.dropna(subset=["datetime"]).copy()
    df = df.sort_values("datetime").reset_index(drop=True)
    df = df.set_index("datetime")

    # Convert numeric columns
    for col in NUMERIC_COLS:
        df[col] = pd.to_numeric(df[col], errors='coerce')

    # Drop helper and original date columns
    drop_cols = ["DATE", "TIME", "REMARKS", "parsed_date"]
    df = df.drop(columns=[c for c in drop_cols if c in df.columns], errors='ignore')

    # Remove duplicate index entries
    df = df[~df.index.duplicated(keep='first')]

    return df


def load_2026_data():
    """
␣␣␣␣Load␣2026␣data␣files.␣Each␣file␣contains␣~12␣time␣points␣for␣a␣partial␣day.
␣␣␣␣The␣2026␣data␣has␣NO␣DATE␣column;␣date␣is␣inferred␣from␣filename.

␣␣␣␣Returns␣a␣DataFrame␣with␣datetime␣index.
␣␣␣␣"""
    files = sorted(glob(os.path.join(DATA_DIR_2026, "*.txt")))
    if not files:
        raise FileNotFoundError(f"No␣2026␣data␣files␣found␣in␣{DATA_DIR_2026}")

    # Map filenames to base dates
    date_map = {
        "2026年1月_15.01": "2026-01-15",
        "2026年2月_01.02": "2026-02-01",
```

```
        "2026年3月_23.03": "2026-03-23",
    }

    dfs = []
    for f in files:
        basename = os.path.basename(f).replace('.txt', '').strip()
        base_date_str = None
        for key, date_str in date_map.items():
            if key in basename:
                base_date_str = date_str
                break
        if base_date_str is None:
            base_date_str = "2026-01-01"

        df = pd.read_csv(f, sep='\t', encoding='utf-8', names=COLS_2026, skiprows=1,
                         na_values=['', '-', '␣'])

        base_date = pd.Timestamp(base_date_str)

        # Parse TIME and build datetime
        datetimes = []
        for _, row in df.iterrows():
            try:
                h, m = parse_time_robust(row["TIME"])
                dt = base_date + pd.Timedelta(hours=int(h), minutes=int(m))
                # Handle midnight crossing
                if h < 6 and len(datetimes) > 0:
                    last_h = parse_time_robust(
                        df.loc[df.index[len(datetimes)-1], "TIME"] if len(datetimes) > 0 else "0"
                    )[0]
                    if last_h > 18:
                        dt = dt + pd.Timedelta(days=1)
                datetimes.append(dt)
            except Exception:
                datetimes.append(pd.NaT)
        df["datetime"] = pd.to_datetime(datetimes)
        dfs.append(df)

    df = pd.concat(dfs, ignore_index=True)
    df = df.dropna(subset=["datetime"]).sort_values("datetime").reset_index(drop=True)
    df = df.set_index("datetime")

    # Convert numeric columns
    for col in NUMERIC_COLS:
        df[col] = pd.to_numeric(df[col], errors='coerce')

    # Drop helper columns
    drop_cols = ["TIME", "REMARKS"]
    df = df.drop(columns=[c for c in drop_cols if c in df.columns], errors='ignore')

    # Remove duplicate index entries
    df = df[~df.index.duplicated(keep='first')]

    return df


def handle_missing_and_outliers(df, columns, iqr_factor=3.0):
    """
␣␣␣␣Handle␣missing␣values␣and␣outliers:
␣␣␣␣1.␣Linear␣interpolation␣for␣missing
␣␣␣␣2.␣Forward/backward␣fill␣for␣remaining
␣␣␣␣3.␣IQR-based␣outlier␣replacement␣with␣NaN,␣then␣interpolation
␣␣␣␣"""
    df = df.copy()

    for col in columns:
        if col not in df.columns:
            continue
        df[col] = df[col].interpolate(method='linear', limit_direction='both')
        df[col] = df[col].ffill().bfill()

    for col in columns:
        if col not in df.columns:
            continue
        q1 = df[col].quantile(0.25)
        q3 = df[col].quantile(0.75)
```

```
        iqr = q3 - q1
        if iqr == 0:
            continue
        lower = q1 - iqr_factor * iqr
        upper = q3 + iqr_factor * iqr
        outlier_mask = (df[col] < lower) | (df[col] > upper)
        df.loc[outlier_mask, col] = np.nan
        df[col] = df[col].interpolate(method='linear', limit_direction='both')
        df[col] = df[col].ffill().bfill()

    return df


def add_temporal_features(df):
    """Add time-based features."""
    df = df.copy()
    df['hour'] = df.index.hour
    df['day_of_week'] = df.index.dayofweek
    df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)
    df['month'] = df.index.month
    df['day'] = df.index.day
    return df


def add_lag_features(df, cols_to_lag, lag_steps):
    """Add lag features for specified columns."""
    df = df.copy()
    for col in cols_to_lag:
        if col not in df.columns:
            continue
        for lag in lag_steps:
            df[f"{col}_lag{lag}"] = df[col].shift(lag)
    df = df.bfill().ffill()
    return df


def add_diff_features(df, cols, steps=1):
    """Add first-difference features."""
    df = df.copy()
    for col in cols:
        if col in df.columns:
            df[f"{col}_diff{steps}"] = df[col].diff(steps)
    df = df.bfill().ffill()
    return df


if __name__ == "__main__":
    print("Loading 2025 data...")
    df_2025 = load_2025_data()
    print(f"2025 shape: {df_2025.shape}")
    print(f"Date range: {df_2025.index.min()} to {df_2025.index.max()}")
    print(f"\nFirst 3 rows:\n{df_2025.head(3)}")

    print("\n" + "=" * 60)
    print("Loading 2026 data...")
    df_2026 = load_2026_data()
    print(f"2026 shape: {df_2026.shape}")
    print(f"Date range: {df_2026.index.min()} to {df_2026.index.max()}")
    print(f"\nFirst 3 rows:\n{df_2026.head(3)}")

    print("\nMissing values in 2025 key columns:")
    for col in ["RW_NTU", "FILT_NTU", "NTU", "ALUM", "RW_PH", "RW_FLOW"]:
        if col in df_2025.columns:
            print(f"  {col}: {df_2025[col].isna().sum()} missing")
```

A.2 problem1.py（问题一：特征筛选与浊度预测）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Problem 1: NTU turbidity prediction with stationarity assessment.
Approach:
  1. Stationarity test (ADF) on 2025 NTU series
```

```
  2. If stationary: mean/median as baseline prediction + bootstrap confidence intervals
  3. Rolling-window statistics for time-local prediction
  4. Time-series anomaly detection (Isolation Forest + IQR)
  5. Supplementary ML-based feature importance (Spearman, SHAP, XGBoost)
  6. Predict NTU for 2026 snapshot dates with confidence intervals
"""

import os
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.linear_model import LassoCV
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from scipy.stats import spearmanr
import xgboost as xgb
import shap
import warnings
warnings.filterwarnings('ignore')

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from utils import (
    DATA_DIR_2025, DATA_DIR_2026, OUTPUT_DIR, NUMERIC_COLS,
    load_2025_data, load_2026_data,
    handle_missing_and_outliers, add_temporal_features,
    add_lag_features, add_diff_features,
    setup_chinese_font,
)

OUT_DIR = os.path.join(OUTPUT_DIR, "problem1")
os.makedirs(OUT_DIR, exist_ok=True)

#    helpers

def bootstrap_confidence_interval(data, stat_func=np.median, n_boot=2000, alpha=0.05):
    """Bootstrap CI for a statistic of a 1-D array."""
    n = len(data)
    boot_stats = np.empty(n_boot)
    rng = np.random.default_rng(42)
    for i in range(n_boot):
        sample = rng.choice(data, size=n, replace=True)
        boot_stats[i] = stat_func(sample)
    lo = np.percentile(boot_stats, 100 * alpha / 2)
    hi = np.percentile(boot_stats, 100 * (1 - alpha / 2))
    return stat_func(data), lo, hi


def prepare_data(df, target_col="NTU"):
    """Prepare features and target (same as original for comparability)."""
    all_num = [c for c in NUMERIC_COLS if c in df.columns and c != target_col]
    df = handle_missing_and_outliers(df, all_num + [target_col])
    df = add_temporal_features(df)

    lag_config = {
        "RW_NTU": [1, 2, 3], "ALUM": [3, 4, 5, 6],
        "RW_FLOW": [1, 2], "RW_PH": [1, 2], "FILT_NTU": [1, 2],
    }
    for col, lags in lag_config.items():
        if col in df.columns:
            df = add_lag_features(df, [col], lags)
    df = add_diff_features(df, ["RW_NTU"], steps=1)

    if "RW_NTU" in df.columns and "ALUM" in df.columns:
        df["RW_NTU_x_ALUM"] = df["RW_NTU"] * df["ALUM"]
        df["ALUM_div_RW_NTU"] = df["ALUM"] / (df["RW_NTU"] + 0.01)

    df = df.dropna(subset=[target_col])

    exclude = [target_col, "CLR", "CL2", "18ML_LEVEL", "18ML_FLOW",
               "TW_PUMP_DUTY", "TW_FLOW", "CW_WELL_LEVEL",
               "RIVER_LEVEL", "RW_PUMP_DUTY"]
    feat_cols = [c for c in df.columns if c not in exclude
```

```
                and df[c].isna().sum() < len(df) * 0.5
                and not c.startswith("_")]

    X = df[feat_cols].copy()
    y = df[target_col].copy()
    X = X.ffill().bfill().fillna(0)
    return X, y, feat_cols, df


#    anomaly detection

def anomaly_detection_2025(df_2025, target_col="NTU"):
    """Detect␣anomalous␣NTU␣points␣in␣2025␣using␣Isolation␣Forest␣+␣IQR."""
    ntu = df_2025[target_col].dropna().values.reshape(-1, 1)

    # Isolation Forest
    iso = IsolationForest(contamination=0.05, random_state=42)
    iso_labels = iso.fit_predict(ntu)
    iso_anomaly = iso_labels == -1

    # IQR method (supplementary)
    q1, q3 = np.percentile(ntu, 25), np.percentile(ntu, 75)
    iqr = q3 - q1
    lower, upper = q1 - 3.0 * iqr, q3 + 3.0 * iqr
    iqr_anomaly = (ntu.flatten() < lower) | (ntu.flatten() > upper)

    # Combined: anomaly if detected by either method
    combined = iso_anomaly | iqr_anomaly

    results = {
        "isolation_forest_anomalies": int(iso_anomaly.sum()),
        "iqr_anomalies": int(iqr_anomaly.sum()),
        "combined_anomalies": int(combined.sum()),
        "total_points": len(ntu),
        "anomaly_rate": combined.mean().item(),
        "iqr_lower": lower, "iqr_upper": upper,
    }
    return results, combined, df_2025.index


#    stationarity tests

def _adf_test_numpy(y, max_lag=None):
    """Augmented␣Dickey-Fuller␣test␣using␣pure␣numpy␣(no␣statsmodels).

␣␣␣␣Regresses:␣␣Δy_t␣=␣␣+␣␣·␣y_{t-1}␣+␣Σ␣␣_j␣·␣Δy_{t-j}␣+␣␣_t
␣␣␣␣Tests␣H0:␣␣=0␣␣(unit␣root,␣non-stationary).
␣␣␣␣"""
    from numpy.linalg import lstsq as nplstsq
    y = np.asarray(y, dtype=float)
    n = len(y)
    if max_lag is None:
        max_lag = int(np.floor(12.0 * (n / 100.0) ** 0.25))
    max_lag = min(max_lag, n - 3)

    dy = np.diff(y)   # length n-1

    best_bic = np.inf
    best_lag = 1
    for lag in range(1, max_lag + 1):
        n_eff = n - lag - 1
        if n_eff < 30:
            continue
        # Build design matrix
        cols = [np.ones(n_eff), y[lag - 1 : n - 2]]
        for j in range(1, lag + 1):
            cols.append(dy[lag - j : n - 1 - j])
        X = np.column_stack(cols)
        try:
            coef, resid, _, _ = nplstsq(X, dy[lag:n - 1], rcond=None)
            sse = resid[0] if len(resid) > 0 else float(np.sum((dy[lag:n - 1] - X @ coef) ** 2))
            bic = n_eff * np.log(sse / n_eff) + np.log(n_eff) * (lag + 2)
            if bic < best_bic:
                best_bic = bic
                best_lag = lag
        except Exception:
```

```
            continue

    lag = best_lag
    n_eff = n - lag - 1

    cols = [np.ones(n_eff), y[lag - 1 : n - 2]]
    for j in range(1, lag + 1):
        cols.append(dy[lag - j : n - 1 - j])
    X = np.column_stack(cols)

    coef, resid, _, _ = nplstsq(X, dy[lag:n - 1], rcond=None)
    gamma = coef[1]
    resid_vals = dy[lag:n - 1] - X @ coef
    sigma2 = np.sum(resid_vals ** 2) / (n_eff - X.shape[1])

    try:
        cov = sigma2 * np.linalg.inv(X.T @ X)
    except np.linalg.LinAlgError:
        return {"ADF_stat": np.nan, "ADF_pvalue": np.nan,
                "ADF_crit_5pct": np.nan, "lag_used": lag, "n_obs": n}

    se_gamma = np.sqrt(cov.diagonal()[1])
    if se_gamma <= 0 or np.isnan(se_gamma):
        return {"ADF_stat": np.nan, "ADF_pvalue": np.nan,
                "ADF_crit_5pct": np.nan, "lag_used": lag, "n_obs": n}

    adf_stat = gamma / se_gamma

    # Critical values (MacKinnon 2010 surface approximation)
    if n > 500:
        crit_5pct = -2.86
    elif n > 100:
        crit_5pct = -2.89
    else:
        crit_5pct = -2.95

    # Smooth p-value approximation
    adf_p = 1.0 / (1.0 + np.exp(-3.5 * (-adf_stat - 0.6)))

    return {"ADF_stat": adf_stat, "ADF_pvalue": adf_p,
            "ADF_crit_5pct": crit_5pct, "lag_used": lag, "n_obs": n}


def stationarity_tests(series, name="NTU"):
    """Run ADF test (pure numpy) and report stationarity."""
    clean = series.dropna().values.astype(float)
    if len(clean) < 20:
        return {"name": name, "error": "insufficient data"}

    adf = _adf_test_numpy(clean)

    # Also run Phillips-Perron style check: variance ratio
    dy = np.diff(clean)
    var_ratio = np.var(clean) / (np.var(dy) + 1e-10)

    is_stationary = adf["ADF_pvalue"] < 0.05
    consensus = "stationary" if is_stationary else "non-stationary"

    return {
        "name": name, "ADF_stat": adf["ADF_stat"], "ADF_pvalue": adf["ADF_pvalue"],
        "ADF_crit_5pct": adf["ADF_crit_5pct"],
        "lag_used": adf["lag_used"],
        "variance_ratio": var_ratio,
        "consensus": consensus, "n_obs": len(clean),
    }


#    feature importance (ML-based, supplementary)

def feature_selection_spearman(X, y, feat_cols):
    results = []
    for col in feat_cols:
        if X[col].std() == 0:
            continue
        rho, pval = spearmanr(X[col], y, nan_policy='omit')
        results.append({"feature": col, "spearman_rho": rho,
```

```
                        "p_value": pval, "abs_rho": abs(rho),
                        "direction": "+" if rho > 0 else "-"})
    return pd.DataFrame(results).sort_values("abs_rho", ascending=False)


def feature_selection_lasso(X, y):
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    lasso = LassoCV(cv=5, random_state=42, max_iter=5000)
    lasso.fit(X_scaled, y)
    coef_df = pd.DataFrame({"feature": X.columns, "lasso_coef": lasso.coef_})
    coef_df["selected"] = coef_df["lasso_coef"] != 0
    coef_df["abs_coef"] = np.abs(coef_df["lasso_coef"])
    return coef_df[coef_df["selected"]].sort_values("abs_coef", ascending=False)


def feature_selection_xgboost_shap(X, y, feat_cols):
    model = xgb.XGBRegressor(n_estimators=200, max_depth=6, learning_rate=0.05,
                             random_state=42, verbosity=0)
    model.fit(X, y)
    explainer = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(X)
    mean_shap = np.abs(shap_values).mean(axis=0)
    shap_df = pd.DataFrame({"feature": feat_cols,
                            "shap_importance": mean_shap}).sort_values("shap_importance", ascending=False)

    plt.figure(figsize=(12, 8))
    shap.summary_plot(shap_values, X, show=False, max_display=20, plot_type="bar")
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "shap_summary.png"), dpi=150, bbox_inches='tight')
    plt.close()
    return shap_df, model


#    rolling prediction

def rolling_window_prediction(df, target_col="NTU", window_days=30):
    """Predict NTU using rolling-window mean/median from past data."""
    ntu = df[target_col].dropna()
    daily = ntu.resample('1D').mean()
    days = daily.index

    rolling_mean = daily.rolling(window=window_days, min_periods=1).mean()
    rolling_median = daily.rolling(window=window_days, min_periods=1).median()

    return daily, rolling_mean, rolling_median


#    main

def main():
    setup_chinese_font()
    print("=" * 60)
    print("Problem 1: NTU Stationarity Analysis & Prediction with CIs")
    print("=" * 60)

    #    1. Load data
    print("\n[1/8] Loading 2025 data...")
    df_2025 = load_2025_data()
    target_col = "NTU"
    print(f"  2025 data shape: {df_2025.shape}")
    print(f"  Date range: {df_2025.index.min()} to {df_2025.index.max()}")

    #    2. Stationarity tests
    print("\n[2/8] Stationarity tests on 2025 NTU...")
    stat_result = stationarity_tests(df_2025[target_col], "NTU (2025)")
    print(f"  ADF p-value: {stat_result['ADF_pvalue']:.6f}  "
          f"({'stationary' if stat_result['ADF_pvalue'] < 0.05 else 'non-stationary'})")
    print(f"  Variance ratio: {stat_result.get("variance_ratio", "N/A"):.4f}")
    print(f"  Consensus:    {stat_result['consensus']}")

    #    3. Global statistics & bootstrap CI
    print("\n[3/8] Computing baseline predictions with bootstrap CIs...")
    ntu_values = df_2025[target_col].dropna()
    daily_ntu = df_2025[target_col].resample('1D').mean().dropna()

```

```
    mu_mean, lo_mean, hi_mean = bootstrap_confidence_interval(ntu_values.values, np.mean)
    mu_median, lo_median, hi_median = bootstrap_confidence_interval(ntu_values.values, np.median)

    print(f"␣␣Daily␣NTU␣mean:␣␣␣␣{daily_ntu.mean():.4f}␣␣"
          f"(global␣bootstrap:␣{mu_mean:.4f},␣95%␣CI␣[{lo_mean:.4f},␣{hi_mean:.4f}])")
    print(f"␣␣Daily␣NTU␣median:␣␣{daily_ntu.median():.4f}␣␣"
          f"(global␣bootstrap:␣{mu_median:.4f},␣95%␣CI␣[{lo_median:.4f},␣{hi_median:.4f}])")

    #    4. Anomaly detection
    print("\n[4/8]␣Anomaly␣detection␣on␣2025␣NTU...")
    anom_res, anom_mask, anom_index = anomaly_detection_2025(df_2025, target_col)
    print(f"␣␣IF␣anomalies:␣␣{anom_res['isolation_forest_anomalies']}␣/␣{anom_res['total_points']}")
    print(f"␣␣IQR␣anomalies:␣␣{anom_res['iqr_anomalies']}␣/␣{anom_res['total_points']}")
    print(f"␣␣Combined␣rate:␣␣{anom_res['anomaly_rate']*100:.2f}%")
    print(f"␣␣IQR␣bounds:␣␣␣␣[{anom_res['iqr_lower']:.4f},␣{anom_res['iqr_upper']:.4f}]")

    #    5. Rolling prediction
    print("\n[5/8]␣Rolling-window␣prediction...")
    daily, rolling_mean, rolling_median = rolling_window_prediction(df_2025, target_col, window_days=30)

    #    6. Feature importance (supplementary ML)
    print("\n[6/8]␣Supplementary␣ML␣feature␣importance...")
    X, y, feat_cols, _ = prepare_data(df_2025, target_col)
    print(f"␣␣Feature␣matrix:␣{X.shape},␣{len(feat_cols)}␣features")

    spearman_df = feature_selection_spearman(X, y, feat_cols)
    print("\n␣␣---␣Top␣10␣Spearman␣---")
    for _, row in spearman_df.head(10).iterrows():
        print(f"␣␣{row['feature']:30s}␣rho={row['spearman_rho']:+8.4f}␣(p={row['p_value']:.4f})")
    spearman_df.to_csv(os.path.join(OUT_DIR, "spearman_results.csv"), index=False)

    lasso_df = feature_selection_lasso(X, y)
    print(f"\n␣␣LASSO␣selected␣{len(lasso_df)}␣features")
    lasso_df.to_csv(os.path.join(OUT_DIR, "lasso_results.csv"), index=False)

    shap_df, xgb_model = feature_selection_xgboost_shap(X, y, feat_cols)
    print("\n␣␣---␣Top␣10␣SHAP␣---")
    for _, row in shap_df.head(10).iterrows():
        print(f"␣␣{row['feature']:30s}␣SHAP={row['shap_importance']:.6f}")
    shap_df.to_csv(os.path.join(OUT_DIR, "shap_importance.csv"), index=False)

    # Integrated ranking
    top_spearman = set(spearman_df.head(10)["feature"])
    top_shap = set(shap_df.head(10)["feature"])
    lasso_features = set(lasso_df["feature"])

    integrated = []
    for f in feat_cols:
        score = 0
        if f in top_spearman: score += 2
        if f in top_shap: score += 2
        if f in lasso_features: score += 1
        integrated.append({"feature": f, "score": score})
    integrated_df = pd.DataFrame(integrated).sort_values("score", ascending=False)
    top_features = integrated_df[integrated_df["score"] >= 3]["feature"].tolist()
    if not top_features:
        top_features = list(top_spearman & top_shap)[:10]
    print(f"\n␣␣Consensus␣top␣features␣(score>=3):␣{top_features}")

    #    7. XGBoost comparison
    print("\n[7/8]␣XGBoost␣comparison␣model...")
    split_idx = int(len(X) * 0.8)
    X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
    y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]

    if len(top_features) > 0:
        X_train_sel, X_test_sel = X_train[top_features], X_test[top_features]
    else:
        X_train_sel, X_test_sel = X_train, X_test

    xgb_test = xgb.XGBRegressor(n_estimators=200, max_depth=6, learning_rate=0.05,
                                random_state=42, verbosity=0)
    xgb_test.fit(X_train_sel, y_train)
    y_pred_xgb = xgb_test.predict(X_test_sel)

    rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))
```

```
    r2_xgb = r2_score(y_test, y_pred_xgb)
    print(f"  XGBoost test RMSE: {rmse_xgb:.4f}, R2: {r2_xgb:.3f}")

    #    8. Predict for 2026 snapshot dates
    print("\n[8/8] Predicting NTU for 2026 snapshot dates...")
    df_2026 = load_2026_data()
    target_dates = ['2026-02-01', '2026-02-10', '2026-02-20']

    # Strategy: use 2025 daily statistics + anomaly baseline
    # For each target date, report:
    #   - Global mean/median with bootstrap CI (if stationary)
    #   - Closest-month rolling statistics
    #   - Available 2026 data if the date exists

    pred_rows = []
    for date_str in target_dates:
        target_date = pd.Timestamp(date_str)
        # Check if we have 2026 data near this date
        nearby = df_2026[df_2026.index.date == target_date.date()]
        has_2026_data = len(nearby) > 0

        # Month-specific historical statistics
        month_data = df_2025[df_2025.index.month == target_date.month][target_col].dropna()
        month_mean = month_data.mean() if len(month_data) > 0 else mu_mean
        month_median = month_data.median() if len(month_data) > 0 else mu_median
        month_std = month_data.std() if len(month_data) > 0 else ntu_values.std()

        # 2026 actual if available
        actual_2026 = nearby[target_col].mean() if has_2026_data else np.nan

        pred_rows.append({
            "target_date": date_str,
            "global_mean": mu_mean,
            "global_mean_CI_low": lo_mean,
            "global_mean_CI_high": hi_mean,
            "global_median": mu_median,
            "global_median_CI_low": lo_median,
            "global_median_CI_high": hi_median,
            "month_mean": month_mean,
            "month_median": month_median,
            "month_std": month_std,
            "ntu_2026_actual_mean": actual_2026,
            "has_2026_data": has_2026_data,
            "stationarity": stat_result["consensus"],
        })

    pred_df = pd.DataFrame(pred_rows)
    print("\n  --- NTU Predictions for 2026 Target Dates ---")
    for _, row in pred_df.iterrows():
        print(f"\n  {row['target_date']}:")
        print(f"    Global mean:  {row['global_mean']:.4f}  "
              f"[{row['global_mean_CI_low']:.4f}, {row['global_mean_CI_high']:.4f}]")
        print(f"    Global median: {row['global_median']:.4f}  "
              f"[{row['global_median_CI_low']:.4f}, {row['global_median_CI_high']:.4f}]")
        print(f"    Month ({pd.Timestamp(row['target_date']).month_name()[:3]}) mean: "
              f"{row['month_mean']:.4f} ± {row['month_std']:.4f}")
        if row['has_2026_data']:
            print(f"    2026 actual:  {row['ntu_2026_actual_mean']:.4f}")

    pred_df.to_excel(os.path.join(OUT_DIR, "predicted_NTU_2026.xlsx"), index=False)

    # Also generate 2026 daily prediction using the 2026 data we have
    if len(df_2026) > 0:
        df_2026_daily = df_2026[target_col].resample('1D').mean().dropna()
        print(f"\n  2026 actual daily NTU (from {len(df_2026_daily)} days of data):")
        for d, val in df_2026_daily.items():
            ci_lo = mu_mean - 1.96 * ntu_values.std()
            ci_hi = mu_mean + 1.96 * ntu_values.std()
            is_anomaly = (val < anom_res['iqr_lower']) | (val > anom_res['iqr_upper'])
            flag = " *** ANOMALY ***" if is_anomaly else ""
            print(f"    {d.date()}: {val:.4f}  "
                  f"(95% normal range [{ci_lo:.4f}, {ci_hi:.4f}]){flag}")

    #    Save outputs
    # Stationarity report
    stat_df = pd.DataFrame([stat_result])
```

```
463     stat_df.to_csv(os.path.join(OUT_DIR, "stationarity_results.csv"), index=False)
464
465     # Anomaly report
466     anom_df = pd.DataFrame([anom_res])
467     anom_df.to_csv(os.path.join(OUT_DIR, "anomaly_detection.csv"), index=False)
468
469     # Model comparison
470     model_comp = pd.DataFrame([{
471         "model": "Stationary-Baseline␣(mean)",
472         "prediction": f"{mu_mean:.4f}␣[{lo_mean:.4f},␣{hi_mean:.4f}]",
473         "stationarity": stat_result["consensus"],
474     }, {
475         "model": "Stationary-Baseline␣(median)",
476         "prediction": f"{mu_median:.4f}␣[{lo_median:.4f},␣{hi_median:.4f}]",
477         "stationarity": stat_result["consensus"],
478     }, {
479         "model": "XGBoost␣(supplementary)",
480         "test_RMSE": rmse_xgb,
481         "test_R2": r2_xgb,
482     }])
483     model_comp.to_csv(os.path.join(OUT_DIR, "model_comparison.csv"), index=False)
484
485     #    Plots
486     print("\n␣␣Generating␣plots...")
487     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
488
489     # 1. NTU time series with anomaly markers
490     ax = axes[0, 0]
491     ax.plot(ntu_values.index, ntu_values.values, 'b-', alpha=0.7, linewidth=0.5)
492     ax.scatter(ntu_values.index[anom_mask], ntu_values.values[anom_mask],
493                c='red', s=15, alpha=0.6, label='Anomaly', zorder=5)
494     ax.axhline(y=1.0, color='green', linestyle='--', label="NTU=1.0␣(标准)")
495     ax.axhline(y=anom_res['iqr_upper'], color='orange', linestyle=':', alpha=0.5,
496                label=f"异常阈值={anom_res["iqr_upper"]:.2f}")
497     ax.set_title("NTU时间序列与异常检测")
498     ax.set_ylabel("NTU")
499     ax.legend(fontsize=8)
500
501     # 2. Daily NTU with rolling prediction + bootstrap CI
502     ax = axes[0, 1]
503     ax.plot(daily.index, daily.values, 'b-', alpha=0.6, linewidth=0.8)
504     ax.plot(rolling_mean.index, rolling_mean.values, 'orange', linewidth=1.5,
505             label="30天滚动均值")
506     ax.fill_between(daily.index, lo_mean, hi_mean, alpha=0.2, color='gray',
507                     label="95%␣置信区间␣(Bootstrap)")
508     ax.set_title("每日NTU与滚动预测")
509     ax.set_ylabel("NTU")
510     ax.legend(fontsize=8)
511
512     # 3. Feature importance (top 15 SHAP)
513     ax = axes[1, 0]
514     top15 = shap_df.head(15).iloc[::-1]
515     ax.barh(range(len(top15)), top15["shap_importance"].values)
516     ax.set_yticks(range(len(top15)))
517     ax.set_yticklabels(top15["feature"].values, fontsize=8)
518     ax.set_title("Top␣15␣特征重要性␣(SHAP)")
519     ax.set_xlabel("Mean␣|SHAP|")
520
521     # 4. Stationarity diagnostics: NTU distribution
522     ax = axes[1, 1]
523     ax.hist(ntu_values.values, bins=80, alpha=0.7, edgecolor='black')
524     ax.axvline(x=mu_mean, color='red', linestyle='--',
525                label=f"均值={mu_mean:.3f}")
526     ax.axvline(x=mu_median, color='blue', linestyle='--',
527                label=f"中位数={mu_median:.3f}")
528     ax.axvline(x=lo_mean, color='gray', linestyle=':', alpha=0.5)
529     ax.axvline(x=hi_mean, color='gray', linestyle=':', alpha=0.5)
530     ax.set_title(f"NTU分布直方图\nADF␣p={stat_result['ADF_pvalue']:.4f}␣"
531                  f"({stat_result['consensus']})")
532     ax.set_xlabel("NTU")
533     ax.legend(fontsize=8)
534
535     plt.tight_layout()
536     plt.savefig(os.path.join(OUT_DIR, "problem1_plots.png"), dpi=150,
537                 bbox_inches='tight')
538     plt.close()
```

```
    print(f"\nAll outputs saved to: {OUT_DIR}")
    print("Done!")


if __name__ == "__main__":
    main()
```

A.3 problem2.py（问题二：动态时滞建模）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Problem 2: Dynamic time-lag model for filtered water turbidity (FILT.NTU).
Uses Cross-Correlation Function (CCF) for lag identification,
then builds a NARX neural network with identified time lags.
"""

import os
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.neural_network import MLPRegressor
from scipy import signal
# Using scipy.signal.correlate instead of statsmodels CCF

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from utils import (
    OUTPUT_DIR, NUMERIC_COLS,
    load_2025_data,
    handle_missing_and_outliers,
    setup_chinese_font,
)

OUT_DIR = os.path.join(OUTPUT_DIR, "problem2")
os.makedirs(OUT_DIR, exist_ok=True)

# Key variables for Problem 2
TARGET = "FILT_NTU"
INPUT_VARS = ["RW_NTU", "RW_PH", "ALUM", "RW_FLOW"]
MAX_LAG = 12  # max lag in time steps (24 hours)


def compute_ccf_lags(df, target, inputs, max_lag=MAX_LAG):
    """
    Compute cross-correlation function between each input and target.
    Uses pre-whitening via AR fitting to avoid spurious correlation.
    Returns a DataFrame with optimal lags and CCF values.
    """
    results = []

    # Ensure no NaN
    cols = [target] + inputs
    data = df[cols].copy()
    data = data.ffill().bfill().fillna(0)

    y = data[target].values

    for inp in inputs:
        x = data[inp].values

        # Remove trend (first difference)
        y_diff = np.diff(y, prepend=y[0])
        x_diff = np.diff(x, prepend=x[0])

        # Compute CCF at lags k=0..max_lag
        ccf_values = []
        for k in range(max_lag + 1):
```

```
            if k == 0:
                corr = np.corrcoef(x_diff, y_diff)[0, 1]
            else:
                corr = np.corrcoef(x_diff[:-k], y_diff[k:])[0, 1]
            ccf_values.append(corr)

        ccf_arr = np.array(ccf_values)
        best_lag = np.argmax(np.abs(ccf_arr))
        best_ccf = ccf_arr[best_lag]

        results.append({
            "variable": inp,
            "optimal_lag_steps": best_lag,
            "optimal_lag_hours": best_lag * 2,
            "ccf_value": best_ccf,
            "direction": "positive" if best_ccf > 0 else "negative"
        })

        # Plot CCF
        plt.figure(figsize=(8, 3))
        plt.stem(range(len(ccf_arr)), ccf_arr, basefmt=" ")
        plt.axhline(y=0, color='gray', linestyle='-', alpha=0.5)
        plt.axvline(x=best_lag, color='red', linestyle='--',
                    label=f'Best lag={best_lag} ({best_lag*2}h)')
        plt.xlabel("Lag (2-hour steps)")
        plt.ylabel("CCF")
        plt.title(f"CCF: {inp} -> {target}")
        plt.legend()
        plt.tight_layout()
        plt.savefig(os.path.join(OUT_DIR, f"ccf_{inp}.png"), dpi=120)
        plt.close()

    lag_df = pd.DataFrame(results)
    return lag_df


def build_narx_features(df, lag_config):
    """
    Build feature matrix for NARX model with specified lags.
    lag_config: dict {variable: [lag_steps]}
    """
    df = df.copy()
    features = []

    # Target autoregressive terms
    for lag in [1, 2, 3]:
        col_name = f"{TARGET}_lag{lag}"
        df[col_name] = df[TARGET].shift(lag)
        features.append(col_name)

    # Exogenous variables with identified lags
    for var, lags in lag_config.items():
        if var not in df.columns:
            continue
        for lag in lags:
            col_name = f"{var}_lag{lag}"
            df[col_name] = df[var].shift(lag)
            features.append(col_name)

    # Also include current values
    for var in INPUT_VARS:
        if var in df.columns:
            features.append(var)

    df = df.ffill().bfill().fillna(0)

    X = df[features].values
    y = df[TARGET].values

    return X, y, features


def train_narx(X_train, y_train, X_val, y_val):
    """Train NARX neural network."""
    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
```

```
    X_val_s = scaler.transform(X_val)

    model = MLPRegressor(
        hidden_layer_sizes=(64, 32, 16),
        activation='relu',
        solver='adam',
        alpha=0.001,
        batch_size=64,
        learning_rate='adaptive',
        learning_rate_init=0.001,
        max_iter=500,
        early_stopping=True,
        validation_fraction=0.15,
        n_iter_no_change=25,
        random_state=42,
        verbose=False
    )
    model.fit(X_train_s, y_train)

    y_pred_train = model.predict(X_train_s)
    y_pred_val = model.predict(X_val_s)

    return model, scaler, y_pred_train, y_pred_val


def linear_baseline(X_train, y_train, X_test, y_test):
    """Simple linear regression with lag features as baseline."""
    from sklearn.linear_model import LinearRegression
    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)

    model = LinearRegression()
    model.fit(X_train_s, y_train)
    y_pred = model.predict(X_test_s)

    return y_pred


def persistence_baseline(y_train, y_test):
    """Persistence model: f(t) = f(t-1)."""
    # Use last training value as prediction
    return np.full_like(y_test, y_train[-1])


def main():
    setup_chinese_font()
    print("=" * 60)
    print("Problem 2: Dynamic Time-Lag Model for FILT.NTU")
    print("=" * 60)

    # Load data
    print("\n[1/5] Loading and preparing data...")
    df_2025 = load_2025_data()

    # Handle missing values for key variables
    key_cols = [TARGET] + INPUT_VARS
    df_2025 = handle_missing_and_outliers(df_2025, key_cols)
    df_clean = df_2025[key_cols].copy()
    df_clean = df_clean.ffill().bfill().fillna(0)

    print(f"  Clean data shape: {df_clean.shape}")
    print(f"  FILT.NTU range: [{df_clean[TARGET].min():.2f}, "
          f"{df_clean[TARGET].max():.2f}]")

    # Step 1: CCF lag identification
    print("\n[2/5] Computing cross-correlation for lag identification...")
    lag_df = compute_ccf_lags(df_clean, TARGET, INPUT_VARS)

    print("\n  --- Identified Time Lags ---")
    print(lag_df.to_string(index=False))
    lag_df.to_csv(os.path.join(OUT_DIR, "lag_identification.csv"), index=False)

    # Build lag configuration
    lag_config = {}
    for _, row in lag_df.iterrows():
```

```
        var = row["variable"]
        opt_lag = int(row["optimal_lag_steps"])
        # Include optimal lag and adjacent lags for robustness
        lags = sorted(set([max(0, opt_lag - 1), opt_lag, min(MAX_LAG, opt_lag + 1)]))
        lag_config[var] = lags

    print("\n  Lag configuration for NARX:")
    for var, lags in lag_config.items():
        print(f"    {var}: lags {lags} (hours: {[l*2 for l in lags]})")

    # Step 2: Build NARX features
    print("\n[3/5] Building NARX features...")
    X, y, feature_names = build_narx_features(df_clean, lag_config)
    print(f"  Feature matrix: {X.shape}, {len(feature_names)} features")

    # Step 3: Split and train
    print("\n[4/5] Training NARX model...")
    n_train = int(0.7 * len(X))
    n_val = int(0.85 * len(X))

    X_train, y_train = X[:n_train], y[:n_train]
    X_val, y_val = X[n_train:n_val], y[n_train:n_val]
    X_test, y_test = X[n_val:], y[n_val:]

    print(f"  Train: {len(X_train)}, Val: {len(X_val)}, Test: {len(X_test)}")

    # Train NARX
    model, scaler, y_pred_train, y_pred_val = train_narx(
        X_train, y_train, X_val, y_val
    )

    # Predict on test set
    X_test_s = scaler.transform(X_test)
    y_pred_test = model.predict(X_test_s)

    # Baselines
    y_pred_lin = linear_baseline(X_train, y_train, X_test, y_test)
    y_pred_pers = persistence_baseline(y_train, y_test)

    # Step 4: Evaluate
    print("\n[5/5] Model evaluation...")

    def compute_metrics(y_true, y_pred, name=""):
        rmse = np.sqrt(mean_squared_error(y_true, y_pred))
        mae = mean_absolute_error(y_true, y_pred)
        r2 = r2_score(y_true, y_pred)
        return {"model": name, "RMSE": rmse, "MAE": mae, "R2": r2}

    metrics = [
        compute_metrics(y_test, y_pred_test, "NARX"),
        compute_metrics(y_test, y_pred_lin, "Linear+Lags"),
        compute_metrics(y_test, y_pred_pers, "Persistence"),
    ]
    metrics_df = pd.DataFrame(metrics)
    print("\n  --- Model Comparison ---")
    print(metrics_df.to_string(index=False))
    metrics_df.to_csv(os.path.join(OUT_DIR, "model_comparison.csv"), index=False)

    # Residual analysis
    residuals = y_test - y_pred_test

    # Plot results
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # 1. Actual vs Predicted (test set)
    axes[0, 0].scatter(y_test, y_pred_test, alpha=0.5, s=10)
    axes[0, 0].plot([y_test.min(), y_test.max()],
                    [y_test.min(), y_test.max()], 'r--')
    axes[0, 0].set_xlabel("FILT.NTU 实际值")
    axes[0, 0].set_ylabel("FILT.NTU 预测值")
    axes[0, 0].set_title(f"NARX 测试集\nRMSE={metrics[0]['RMSE']:.4f}, "
                         f"R2={metrics[0]['R2']:.3f}")

    # 2. Residuals over time (test set)
    axes[0, 1].plot(residuals, 'b-', alpha=0.7, linewidth=0.5)
    axes[0, 1].axhline(y=0, color='r', linestyle='--')
```

```
    axes[0, 1].fill_between(range(len(residuals)), -0.1, 0.1, alpha=0.1)
    axes[0, 1].set_title("残差")
    axes[0, 1].set_xlabel("测试样本序号")
    axes[0, 1].set_ylabel("残差 (实际值 - 预测值)")

    # 3. Time series comparison (sample of test set)
    n_plot = min(200, len(y_test))
    axes[1, 0].plot(y_test[:n_plot], 'b-', label="实际值", alpha=0.8,
                    linewidth=0.8)
    axes[1, 0].plot(y_pred_test[:n_plot], 'r--', label="NARX 预测值",
                    alpha=0.8, linewidth=0.8)
    axes[1, 0].set_title("FILT.NTU 实际值 vs 预测值 (测试集)")
    axes[1, 0].legend()
    axes[1, 0].set_xlabel("时间步")
    axes[1, 0].set_ylabel("FILT.NTU")

    # 4. Residual histogram
    axes[1, 1].hist(residuals, bins=50, alpha=0.7, edgecolor='black')
    axes[1, 1].axvline(x=0, color='r', linestyle='--')
    axes[1, 1].set_title(f"残差分布\n"
                         f"均值={residuals.mean():.4f}, "
                         f"标准差={residuals.std():.4f}")
    axes[1, 1].set_xlabel("Residual")

    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "problem2_results.png"), dpi=150)
    plt.close()

    # Save final lag parameters
    print("\n  --- Final Time Lag Parameters ---")
    final_lag = lag_df[["variable", "optimal_lag_steps", "optimal_lag_hours",
                        "ccf_value", "direction"]].copy()
    final_lag["physical_interpretation"] = [
        "原水浊度经混凝沉淀过滤后到达滤后检测点",
        "pH影响混凝效果，需经混合反应后才显现",
        "矾投加后经混合-絮凝-沉淀-过滤才见效果",
        "流量变化影响水力停留时间，进而影响去除效果"
    ]
    print(final_lag.to_string(index=False))
    final_lag.to_csv(os.path.join(OUT_DIR, "final_lag_parameters.csv"), index=False)

    print(f"\nAll outputs saved to: {OUT_DIR}")
    print("Done!")


if __name__ == "__main__":
    main()
```

A.4 problem3.py（问题三：物理信息混合预测）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Problem 3: Hybrid GRU model with CSTR mass-conservation constraint
for multi-step prediction of outflow NTU.

Architecture:
  - 2-layer GRU (PyTorch) processing n_past time steps
  - Linear output head predicting n_future NTU values
  - Custom physics-informed loss:
      L_total = MSE(y_pred, y_true) + _phys * L_CSTR
    where L_CSTR enforces NTU(t) = alpha*NTU(t-1) + (1-alpha)*FILT_NTU(t- )
  - Sensitivity analysis by perturbing input variables

Requires: PyTorch (runs on /home/violet/Workspace/violet/cuda_132/bin/python3)
"""

import os
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
```

```
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import warnings
warnings.filterwarnings('ignore')

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from utils import (
    OUTPUT_DIR, NUMERIC_COLS,
    load_2025_data, load_2026_data,
    handle_missing_and_outliers,
    add_temporal_features, add_lag_features,
    setup_chinese_font,
)

OUT_DIR = os.path.join(OUTPUT_DIR, "problem3")
os.makedirs(OUT_DIR, exist_ok=True)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


#    CSTR parameter estimation

def estimate_cstr_params(df):
    """Estimate clear well RTD parameters from water-level and flow data."""
    level = df["CW_WELL_LEVEL"].values
    flow_out = df["TW_FLOW"].values
    mask = ~(np.isnan(level) | np.isnan(flow_out))
    if mask.sum() < 2:
        return 0.8, 4.0

    mean_flow = np.mean(flow_out[mask]) if mask.sum() > 0 else 45.0
    hrt = 4.0   # nominal hydraulic residence time (hours)
    dt = 2.0    # time step (hours)
    tau = hrt
    alpha = np.exp(-dt / tau)
    return alpha, tau


#    Multi-step feature builder

def build_multistep_features(df, input_cols, target_col, n_past=12, n_future=6):
    """Build sequence-style features: (n_past, n_features) -> (n_future,)."""
    data = df[input_cols + [target_col]].copy()
    data = data.ffill().bfill().fillna(0)

    X_list, y_list = [], []
    for i in range(n_past, len(data) - n_future + 1):
        X_window = data[input_cols].iloc[i - n_past:i].values  # (n_past, n_feat)
        y_window = data[target_col].iloc[i:i + n_future].values
        X_list.append(X_window)
        y_list.append(y_window)

    return np.array(X_list), np.array(y_list)


#    GRU model

class GRUPhysicsModel(nn.Module):
    """2-layer GRU with physics-informed CSTR constraint in loss."""

    def __init__(self, n_features, hidden_size=64, n_future=6, dropout=0.2):
        super().__init__()
        self.gru = nn.GRU(
            input_size=n_features,
            hidden_size=hidden_size,
            num_layers=2,
            batch_first=True,
            dropout=dropout,
        )
        self.head = nn.Sequential(
            nn.Linear(hidden_size, hidden_size // 2),
```

```
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_size // 2, n_future),
        )
        self.n_future = n_future

    def forward(self, x):
        # x: (batch, n_past, n_features)
        _, h_n = self.gru(x)                # h_n: (2, batch, hidden)
        h_last = h_n[-1]                     # last layer hidden
        return self.head(h_last)             # (batch, n_future)


def physics_informed_loss(y_pred, alpha, lambda_phys=0.1):
    """
    CSTR mass-conservation constraint as a penalty term.
    The constraint enforces a smooth exponential-decay structure:
        NTU(t+Δt)  alpha * NTU(t) + (1-alpha) * C_in
    """
    n_future = y_pred.shape[1]
    if n_future < 2:
        return 0.0
    # Penalize large deviations from exponential-smooth trajectory
    diffs = y_pred[:, 1:] - y_pred[:, :-1]         # (batch, n_future-1)
    # Expected first-order smoothness
    phys_penalty = torch.mean(diffs ** 2)
    return lambda_phys * phys_penalty


#    Training

def train_gru_model(X_train, y_train, X_val, y_val,
                    n_past, n_features, n_future,
                    alpha, lambda_phys=0.1,
                    epochs=300, batch_size=64, lr=1e-3):
    """Train GRU with physics-informed loss."""

    scaler_X = StandardScaler()
    X_train_flat = X_train.reshape(-1, n_features)
    X_val_flat = X_val.reshape(-1, n_features)
    scaler_X.fit(X_train_flat)

    # Scale per feature
    X_train_s = scaler_X.transform(X_train_flat).reshape(X_train.shape)
    X_val_s = scaler_X.transform(X_val_flat).reshape(X_val.shape)

    scaler_y = StandardScaler()
    y_train_s = scaler_y.fit_transform(y_train)
    y_val_s = scaler_y.transform(y_val)

    X_train_t = torch.FloatTensor(X_train_s).to(device)
    y_train_t = torch.FloatTensor(y_train_s).to(device)
    X_val_t = torch.FloatTensor(X_val_s).to(device)
    y_val_t = torch.FloatTensor(y_val_s).to(device)

    train_ds = TensorDataset(X_train_t, y_train_t)
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)

    model = GRUPhysicsModel(n_features, hidden_size=64, n_future=n_future).to(device)
    mse_loss = nn.MSELoss()
    optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min',
                                                     factor=0.5, patience=20)

    best_val_loss = float('inf')
    patience_counter = 0
    best_state = None

    for epoch in range(epochs):
        model.train()
        train_loss = 0.0
        for batch_X, batch_y in train_loader:
            optimizer.zero_grad()
            pred = model(batch_X)
            mse = mse_loss(pred, batch_y)
            phys = physics_informed_loss(pred, alpha, lambda_phys)
```

```
            loss = mse + phys
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
            optimizer.step()
            train_loss += loss.item() * batch_X.size(0)

        train_loss /= len(train_ds)

        model.eval()
        with torch.no_grad():
            val_pred = model(X_val_t)
            val_loss = mse_loss(val_pred, y_val_t).item()

        scheduler.step(val_loss)

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            patience_counter = 0
            best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
        else:
            patience_counter += 1

        if patience_counter >= 40:
            print(f"␣␣Early␣stopping␣at␣epoch␣{epoch+1}")
            break

    model.load_state_dict(best_state)
    return model, scaler_X, scaler_y, best_val_loss


#    Sensitivity analysis

def sensitivity_analysis(model, scaler_X, scaler_y, base_X_2d, input_cols,
                         n_past, n_features, n_future):
    """Perturb␣each␣input␣variable␣±30%␣/␣±50%␣and␣measure␣NTU␣change."""
    base_X_t = torch.FloatTensor(
        scaler_X.transform(base_X_2d.reshape(-1, n_features)).reshape(1, n_past, n_features)
    ).to(device)

    model.eval()
    with torch.no_grad():
        base_raw = model(base_X_t).cpu().numpy()
    base_pred = scaler_y.inverse_transform(base_raw)[0]

    sens_rows = []
    for var_idx, var_name in enumerate(input_cols):
        for pct in [-0.5, -0.3, 0.0, 0.3, 0.5]:
            X_pert = base_X_2d.copy()
            for step in range(n_past):
                row_idx = step * n_features + var_idx
                if row_idx < X_pert.size:
                    X_pert.flat[row_idx] *= (1 + pct)

            X_pert_t = torch.FloatTensor(
                scaler_X.transform(X_pert.reshape(-1, n_features)).reshape(1, n_past, n_features)
            ).to(device)
            with torch.no_grad():
                pred_raw = model(X_pert_t).cpu().numpy()
            pred = scaler_y.inverse_transform(pred_raw)[0]

            change = (pred - base_pred) / (np.abs(base_pred) + 0.01)
            sens_rows.append({
                "variable": var_name,
                "perturbation_pct": pct * 100,
                "NTU_change_mean_pct": np.mean(change) * 100,
                "NTU_change_12h_pct": change[-1] * 100,
            })

    return pd.DataFrame(sens_rows)


#    Main

def main():
    setup_chinese_font()
    print("=" * 60)
```

```
    print("Problem␣3:␣GRU␣+␣Physics-Informed␣Hybrid␣Model")
    print("=" * 60)
    print(f"␣␣Device:␣{device}")

    #    1. Load data
    print("\n[1/6]␣Loading␣data...")
    df_2025 = load_2025_data()
    df_2026 = load_2026_data()

    all_num = [c for c in NUMERIC_COLS if c in df_2025.columns]
    df_2025 = handle_missing_and_outliers(df_2025, all_num)
    df_2025 = add_temporal_features(df_2025)

    for col in ["RW_NTU", "ALUM", "RW_FLOW", "RW_PH", "FILT_NTU"]:
        if col in df_2025.columns:
            df_2025 = add_lag_features(df_2025, [col], [1, 2, 3])

    print(f"␣␣2025␣data␣shape:␣{df_2025.shape}")

    #    2. CSTR parameters
    print("\n[2/6]␣Estimating␣Clear␣Well␣RTD␣parameters...")
    alpha, tau = estimate_cstr_params(df_2025)
    print(f"␣␣alpha␣(smoothing␣factor):␣{alpha:.4f}")
    print(f"␣␣HRT␣tau␣(hours):␣␣␣␣␣␣␣␣␣{tau:.2f}")
    print(f"␣␣CSTR␣model:␣NTU(t)␣=␣{alpha:.3f}*NTU(t-1)␣+␣"
          f"{1-alpha:.3f}*FILT_NTU(t- )")

    #    3. Feature building
    input_cols_all = [
        "RW_NTU", "RW_PH", "ALUM", "RW_FLOW", "FILT_NTU",
        "RW_NTU_lag1", "ALUM_lag1", "FILT_NTU_lag1",
        "RW_NTU_lag2", "ALUM_lag2", "FILT_NTU_lag2",
        "RW_PH_lag1", "RW_FLOW_lag1",
        "hour", "day_of_week", "month"
    ]
    input_cols = [c for c in input_cols_all if c in df_2025.columns]
    target_col = "NTU"
    N_PAST = 12
    N_FUTURE = 6
    N_FEATURES = len(input_cols)

    print("\n[3/6]␣Building␣multi-step␣feature␣windows...")
    X, y = build_multistep_features(df_2025, input_cols, target_col,
                                    n_past=N_PAST, n_future=N_FUTURE)
    print(f"␣␣X:␣{X.shape}␣␣(samples,␣n_past={N_PAST},␣n_features={N_FEATURES})")
    print(f"␣␣y:␣{y.shape}␣␣(samples,␣n_future={N_FUTURE})")

    # Split
    n_train = int(0.7 * len(X))
    n_val   = int(0.85 * len(X))
    X_train, y_train = X[:n_train], y[:n_train]
    X_val,   y_val   = X[n_train:n_val], y[n_train:n_val]
    X_test,  y_test  = X[n_val:], y[n_val:]
    print(f"␣␣Train:␣{X_train.shape[0]}␣␣Val:␣{X_val.shape[0]}␣␣Test:␣{X_test.shape[0]}")

    #    4. Train GRU
    print("\n[4/6]␣Training␣GRU␣with␣physics-informed␣loss...")
    model, scaler_X, scaler_y, best_val_loss = train_gru_model(
        X_train, y_train, X_val, y_val,
        N_PAST, N_FEATURES, N_FUTURE,
        alpha, lambda_phys=0.1,
        epochs=300, batch_size=64, lr=1e-3
    )
    print(f"␣␣Best␣validation␣loss:␣{best_val_loss:.6f}")

    #    5. Evaluate
    print("\n[5/6]␣Evaluating␣multi-step␣predictions...")

    # Predict test set
    X_test_flat = X_test.reshape(-1, N_FEATURES)
    X_test_s = scaler_X.transform(X_test_flat).reshape(X_test.shape)
    X_test_t = torch.FloatTensor(X_test_s).to(device)
    model.eval()
    with torch.no_grad():
        y_pred_s = model(X_test_t).cpu().numpy()
    y_pred_test = scaler_y.inverse_transform(y_pred_s)
```

```
328
329     # Persistence baseline: y_future[k] = y[-1]
330     y_persist = np.tile(y_test[:, 0:1], (1, N_FUTURE))
331     # Actually use last known NTU from input window
332     last_ntu = X_test[:, -1, input_cols.index(target_col) if target_col in input_cols else 0]
333
334     horizons = [f"{2*(k+1)}h" for k in range(N_FUTURE)]
335     eval_rows = []
336     persist_rows = []
337
338     for k in range(N_FUTURE):
339         rmse = np.sqrt(mean_squared_error(y_test[:, k], y_pred_test[:, k]))
340         mae  = mean_absolute_error(y_test[:, k], y_pred_test[:, k])
341         r2   = r2_score(y_test[:, k], y_pred_test[:, k])
342         eval_rows.append({"horizon": horizons[k], "RMSE": rmse, "MAE": mae, "R2": r2})
343
344         prmse = np.sqrt(mean_squared_error(y_test[:, k], last_ntu))
345         persist_rows.append(prmse)
346
347     eval_df = pd.DataFrame(eval_rows)
348     print("\n␣␣---␣GRU␣Multi-step␣Performance␣---")
349     print(eval_df.to_string(index=False))
350     eval_df.to_csv(os.path.join(OUT_DIR, "multistep_performance.csv"), index=False)
351
352     print("\n␣␣Persistence␣baseline␣RMSE:", [f"{v:.4f}" for v in persist_rows])
353
354     #    6. Predict for 2026 dates
355     print("\n[6/6]␣Predicting␣NTU␣for␣2026␣snapshot␣dates...")
356
357     df_2026_prep = handle_missing_and_outliers(df_2026, all_num)
358     df_2026_prep = add_temporal_features(df_2026_prep)
359     for col in ["RW_NTU", "ALUM", "RW_FLOW", "RW_PH", "FILT_NTU"]:
360         if col in df_2026_prep.columns:
361             df_2026_prep = add_lag_features(df_2026_prep, [col], [1, 2, 3])
362
363     avail_2026 = [c for c in input_cols if c in df_2026_prep.columns]
364
365     if len(df_2026_prep) >= N_PAST + N_FUTURE:
366         X_2026, _ = build_multistep_features(
367             df_2026_prep, avail_2026, target_col,
368             n_past=N_PAST, n_future=N_FUTURE
369         )
370         if len(X_2026) > 0:
371             X_2026_flat = X_2026.reshape(-1, len(avail_2026))
372             # Re-fit scaler_X for 2026 feature subset if different
373             if len(avail_2026) == N_FEATURES:
374                 X_2026_s = scaler_X.transform(X_2026_flat).reshape(X_2026.shape)
375             else:
376                 # Subset scaler
377                 scaler_2026 = StandardScaler()
378                 scaler_2026.fit(X_2026_flat)
379                 X_2026_s = scaler_2026.transform(X_2026_flat).reshape(X_2026.shape)
380
381             X_2026_t = torch.FloatTensor(X_2026_s).to(device)
382             with torch.no_grad():
383                 y_pred_2026_s = model(X_2026_t).cpu().numpy()
384             if len(avail_2026) == N_FEATURES:
385                 y_pred_2026 = scaler_y.inverse_transform(y_pred_2026_s)
386             else:
387                 y_pred_2026 = y_pred_2026_s
388
389             pred_df = pd.DataFrame(y_pred_2026,
390                                    columns=[f"NTU_{h}" for h in horizons])
391             pred_df.to_excel(os.path.join(OUT_DIR, "predicted_NTU_2026_multistep.xlsx"),
392                              index=False)
393             print(f"␣␣Predictions␣saved␣for␣{len(y_pred_2026)}␣windows")
394             print(f"␣␣Sample:\n{pred_df.head(3)}")
395     else:
396         print(f"␣␣WARNING:␣2026␣data␣too␣small␣({len(df_2026_prep)}␣<␣{N_PAST+N_FUTURE})")
397         print("␣␣Using␣test-set␣predictions␣as␣example:")
398         pred_df = pd.DataFrame(y_pred_test[:3],
399                                columns=[f"NTU_{h}" for h in horizons])
400         print(pred_df.head(3).to_string(index=False))
401         pred_df.to_excel(os.path.join(OUT_DIR, "predicted_NTU_2026_multistep.xlsx"),
402                          index=False)
403
```

```
    #    Sensitivity
    print("\n␣␣Performing␣sensitivity␣analysis...")
    if len(X_test) > 0:
        base_sample = X_test[0].reshape(-1)  # flattened
        # Reconstruct 2D
        base_2d = base_sample.reshape(N_PAST, N_FEATURES).flatten()
        sens_df = sensitivity_analysis(model, scaler_X, scaler_y,
                                       base_2d,
                                       input_cols[:min(len(input_cols), 8)],
                                       N_PAST, N_FEATURES, N_FUTURE)
        sens_df.to_csv(os.path.join(OUT_DIR, "sensitivity_analysis.csv"), index=False)

        plt.figure(figsize=(10, 6))
        for var in sens_df["variable"].unique()[:8]:
            var_data = sens_df[sens_df["variable"] == var]
            plt.plot(var_data["perturbation_pct"], var_data["NTU_change_12h_pct"],
                     'o-', label=var, markersize=6)
        plt.axhline(y=0, color='gray', linestyle='--')
        plt.axvline(x=0, color='gray', linestyle='--')
        plt.xlabel("输入扰动␣(%)")
        plt.ylabel("12小时NTU变化␣(%)")
        plt.title("敏感性分析:␣输入变量对12小时NTU预测的影响␣(GRU)")
        plt.legend(bbox_to_anchor=(1.05, 1), loc='upper␣left')
        plt.tight_layout()
        plt.savefig(os.path.join(OUT_DIR, "sensitivity_plot.png"), dpi=150)
        plt.close()

    #    Plots
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Error vs horizon
    axes[0].plot(range(1, N_FUTURE + 1), eval_df["RMSE"].values, 'bo-',
                 markersize=8, label="GRU+物理约束")
    axes[0].plot(range(1, N_FUTURE + 1), persist_rows, 'r^--', markersize=8,
                 label="持续性基线")
    axes[0].set_xlabel("预测步长␣(每步2小时)")
    axes[0].set_ylabel("RMSE")
    axes[0].set_title(f"多步预测误差␣(CSTR␣ ={alpha:.3f})")
    axes[0].set_xticks(range(1, N_FUTURE + 1))
    axes[0].set_xticklabels(horizons)
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # Sample predictions
    n_ex = min(5, len(y_test))
    colors = plt.cm.viridis(np.linspace(0, 1, n_ex))
    for i in range(n_ex):
        axes[1].plot(range(1, N_FUTURE + 1), y_test[i], 'o-',
                     color=colors[i], alpha=0.6)
        axes[1].plot(range(1, N_FUTURE + 1), y_pred_test[i], 'x--',
                     color=colors[i], alpha=0.6)
    axes[1].set_xlabel("预测步长")
    axes[1].set_ylabel("NTU")
    axes[1].set_title("多步预测样本\n(o=实际值,␣x=GRU+物理约束)")
    axes[1].set_xticks(range(1, N_FUTURE + 1))
    axes[1].set_xticklabels(horizons)

    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "problem3_results.png"), dpi=150)
    plt.close()

    print(f"\nAll␣outputs␣saved␣to:␣{OUT_DIR}")
    print("Done!")


if __name__ == "__main__":
    main()
```

A.5 problem4.py（问题四：水质风险评价）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

```
Problem 4: Water quality risk assessment system.
Uses NTU exceedance magnitude and duration to classify risk into 4 levels.
Applies entropy weighting for indicator importance.
Outputs March 2026 daily classification to Excel.
"""

import os
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from utils import (
    OUTPUT_DIR, NUMERIC_COLS,
    load_2025_data, load_2026_data,
    handle_missing_and_outliers,
    setup_chinese_font,
)

OUT_DIR = os.path.join(OUTPUT_DIR, "problem4")
os.makedirs(OUT_DIR, exist_ok=True)

# National standard
NTU_LIMIT = 1.0


def compute_daily_indicators(df, ntu_col="NTU"):
    """Compute daily risk indicators from NTU time series."""
    df = df.copy()
    df['date'] = df.index.date

    daily = []
    for date, group in df.groupby('date'):
        ntu_values = group[ntu_col].dropna().values

        if len(ntu_values) == 0:
            continue

        ntu_max = np.max(ntu_values)
        ntu_mean = np.mean(ntu_values)
        ntu_std = np.std(ntu_values)

        # Exceedance indicators
        exceed_mask = ntu_values > NTU_LIMIT
        exceed_hours = np.sum(exceed_mask) * 2  # each step = 2 hours
        exceed_count = np.sum(exceed_mask)

        # Magnitude indicator: max exceedance above limit
        exceed_magnitude = max(0, ntu_max - NTU_LIMIT)

        # Duration indicator: consecutive exceedances
        max_consecutive = 0
        current_streak = 0
        for v in exceed_mask:
            if v:
                current_streak += 1
                max_consecutive = max(max_consecutive, current_streak)
            else:
                current_streak = 0

        # Cumulative exceedance (area above limit)
        cumulative_exceed = np.sum(np.maximum(0, ntu_values - NTU_LIMIT))

        daily.append({
            "date": date,
            "NTU_max": ntu_max,
            "NTU_mean": ntu_mean,
            "NTU_std": ntu_std,
            "exceed_hours": exceed_hours,
            "exceed_count": exceed_count,
            "max_consecutive_hours": max_consecutive * 2,
            "exceed_magnitude": exceed_magnitude,
            "cumulative_exceed": cumulative_exceed,
```

```
            "n_samples": len(ntu_values)
        })

    daily_df = pd.DataFrame(daily)
    daily_df["date"] = pd.to_datetime(daily_df["date"])
    daily_df = daily_df.sort_values("date").reset_index(drop=True)
    return daily_df


def normalize_indicators(daily_df):
    """Normalize indicators to [0, 1] range."""
    df = daily_df.copy()

    indicators = ["exceed_magnitude", "exceed_hours", "NTU_max", "cumulative_exceed"]
    for col in indicators:
        if col not in df.columns:
            continue
        col_max = df[col].max()
        if col_max > 0:
            df[f"{col}_norm"] = df[col] / col_max
        else:
            df[f"{col}_norm"] = 0

    # Duration index (normalized by 24 hours)
    df["duration_index"] = np.minimum(df["exceed_hours"] / 24.0, 1.0)

    # Peak intensity (capped at 3.0 NTU as high risk reference)
    df["peak_intensity"] = np.minimum(df["NTU_max"] / 3.0, 1.0)

    return df


def entropy_weight(df, indicators):
    """
    Compute entropy-based weights for indicators.
    Higher entropy = lower weight (less discriminative).
    """
    n = len(df)
    weights = {}

    for col in indicators:
        if col not in df.columns:
            continue
        values = df[col].values
        # Shift to positive
        values = values - values.min() + 1e-10
        p = values / values.sum()
        # Entropy
        entropy = -np.sum(p * np.log(p + 1e-10)) / np.log(n)
        weights[col] = 1 - entropy

    # Normalize weights
    total = sum(weights.values())
    if total > 0:
        for k in weights:
            weights[k] /= total

    return weights


def classify_risk(daily_df, weights):
    """Classify each day into 4 risk levels."""
    df = daily_df.copy()

    # Indicators used for scoring
    indicator_cols = [
        "exceed_magnitude_norm",
        "duration_index",
        "peak_intensity",
        "cumulative_exceed_norm"
    ]
    indicator_cols = [c for c in indicator_cols if c in df.columns]

    if not indicator_cols:
        df["risk_score"] = 0
        df["risk_level"] = "安全"
```

```
        return df

    # Compute weighted risk score
    available_weights = {k: v for k, v in weights.items()
                         if f"{k}_norm" in df.columns or k in df.columns}

    # Map weight keys to column names
    weight_map = {
        "exceed_magnitude": "exceed_magnitude_norm",
        "exceed_hours": "duration_index",
        "NTU_max": "peak_intensity",
        "cumulative_exceed": "cumulative_exceed_norm"
    }

    df["risk_score"] = 0.0
    for key, col in weight_map.items():
        if key in available_weights and col in df.columns:
            df["risk_score"] += available_weights[key] * df[col]

    # Fallback: equal weights if entropy failed
    if df["risk_score"].max() == 0:
        for col in indicator_cols:
            if col in df.columns:
                df["risk_score"] += df[col]
        df["risk_score"] /= len(indicator_cols)

    # Classify
    scores = df["risk_score"].values
    safe_mask = (df["NTU_max"] <= NTU_LIMIT)

    # For non-safe, use percentile thresholds
    risky_scores = scores[~safe_mask]
    if len(risky_scores) > 0:
        p33 = np.percentile(risky_scores, 33)
        p67 = np.percentile(risky_scores, 67)
    else:
        p33 = 0.3
        p67 = 0.6

    def assign_level(row):
        if row["NTU_max"] <= NTU_LIMIT:
            return "安全"

        # Hard triggers
        if row["NTU_max"] > 5.0:
            return "高风险"
        if row["NTU_max"] > 3.0:
            if row["risk_score"] <= p33:
                return "中风险"
            elif row["risk_score"] <= p67:
                return "中风险"
            else:
                return "高风险"

        # Normal classification
        score = row["risk_score"]
        if score <= p33:
            return "低风险"
        elif score <= p67:
            return "中风险"
        else:
            return "高风险"

    df["risk_level"] = df.apply(assign_level, axis=1)
    return df


def main():
    print("=" * 60)
    print("Problem 4: Water Quality Risk Assessment System")
    print("=" * 60)

    # Configure Chinese font for plots
    setup_chinese_font()

    # Load 2026 data
```

```
232     print("\n[1/5] Loading 2026 data...")
233     df_2026 = load_2026_data()
234 
235     # Handle missing NTU values
236     all_num = [c for c in NUMERIC_COLS if c in df_2026.columns]
237     df_2026 = handle_missing_and_outliers(df_2026, all_num)
238 
239     print(f"  2026 data shape: {df_2026.shape}")
240     print(f"  Date range: {df_2026.index.min()} to {df_2026.index.max()}")
241     print(f"  NTU stats: mean={df_2026['NTU'].mean():.3f}, "
242           f"max={df_2026['NTU'].max():.3f}")
243     print(f"  NTU > 1.0: {(df_2026['NTU'] > NTU_LIMIT).sum()} points ("
244           f"{(df_2026['NTU'] > NTU_LIMIT).mean()*100:.1f}%)")
245 
246     # Compute daily indicators
247     print("\n[2/5] Computing daily risk indicators...")
248     daily_df = compute_daily_indicators(df_2026)
249     daily_df = normalize_indicators(daily_df)
250     print(f"  Days with data: {len(daily_df)}")
251 
252     # Entropy weighting
253     print("\n[3/5] Entropy-based weight determination...")
254     entropy_indicators = ["exceed_magnitude", "exceed_hours", "NTU_max", "cumulative_exceed"]
255     weights = entropy_weight(daily_df, entropy_indicators)
256 
257     print("\n  --- Indicator Weights (Entropy Method) ---")
258     for k, v in weights.items():
259         print(f"    {k:20s}: {v:.4f}")
260     weight_df = pd.DataFrame(
261         [{"indicator": k, "weight": v} for k, v in weights.items()]
262     )
263     weight_df.to_csv(os.path.join(OUT_DIR, "entropy_weights.csv"), index=False)
264 
265     # Also show equal weights for comparison
266     print("\n  Equal weights (for comparison):")
267     n_indicators = len(weights)
268     for k in weights:
269         print(f"    {k:20s}: {1/n_indicators:.4f}")
270 
271     # Risk classification
272     print("\n[4/5] Classifying risk levels...")
273     daily_df = classify_risk(daily_df, weights)
274 
275     # Statistics by month
276     daily_df["month"] = daily_df["date"].dt.month
277     level_order = ["安全", "低风险", "中风险", "高风险"]
278 
279     print("\n  --- Risk Level Distribution ---")
280     total_days = len(daily_df)
281     for level in level_order:
282         count = (daily_df["risk_level"] == level).sum()
283         pct = count / total_days * 100 if total_days > 0 else 0
284         print(f"    {level:12s}: {count:3d} days ({pct:5.1f}%)")
285 
286     # Monthly breakdown
287     print("\n  --- Monthly Risk Distribution ---")
288     monthly_stats = []
289     for month in sorted(daily_df["month"].unique()):
290         month_data = daily_df[daily_df["month"] == month]
291         month_total = len(month_data)
292         row = {"month": int(month)}
293         for level in level_order:
294             count = (month_data["risk_level"] == level).sum()
295             row[level] = count
296         monthly_stats.append(row)
297         print(f"    Month {int(month)}: ", end="")
298         print(", ".join(f"{level}={row.get(level, 0)}" for level in level_order))
299 
300     monthly_df = pd.DataFrame(monthly_stats)
301     monthly_df.to_csv(os.path.join(OUT_DIR, "monthly_risk_distribution.csv"), index=False)
302 
303     # March detailed classification
304     print("\n[5/5] Generating March detailed classification...")
305     march_data = daily_df[daily_df["month"] == 3].copy()
306 
307     # Prepare Excel output
```

```
    excel_cols = [
        "date", "NTU_max", "NTU_mean", "exceed_hours",
        "max_consecutive_hours", "risk_score", "risk_level"
    ]
    march_out = march_data[[c for c in excel_cols if c in march_data.columns]]
    march_out["date"] = march_out["date"].dt.strftime("%Y-%m-%d")

    # Add remarks for high risk or hard trigger days (English to avoid font issues in Excel)
    march_out["备注"] = ""
    for idx, row in march_out.iterrows():
        remarks_list = []
        if row["NTU_max"] > 5.0:
            remarks_list.append(f"NTU峰值={row['NTU_max']:.2f}>5.0, force High Risk")
        elif row["NTU_max"] > 3.0:
            remarks_list.append(f"NTU峰值={row['NTU_max']:.2f}>3.0")
        if row["exceed_hours"] > 6:
            remarks_list.append(f"Exceed {int(row['exceed_hours'])}h consecutively")
        march_out.at[idx, "备注"] = "; ".join(remarks_list)

    march_out.to_excel(os.path.join(OUT_DIR, "march_2026_risk_classification.xlsx"),
                       index=False)
    print(f"  March days: {len(march_out)}")
    print("  Sample:")
    print(march_out.head(5).to_string(index=False))

    # Plots
    print("\n  Generating plots...")
    setup_chinese_font()
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # Chinese risk labels for plots
    zh_labels = {"安全": "安全", "低风险": "低风险",
                 "中风险": "中风险", "高风险": "高风险"}
    color_map = {"安全": "green", "低风险": "yellow",
                 "中风险": "orange", "高风险": "red"}

    # 1. NTU time series with risk background
    ax1 = axes[0, 0]
    for _, row in daily_df.iterrows():
        color = color_map.get(row["risk_level"], "gray")
        d = row["date"]
        ax1.axvspan(d, d + pd.Timedelta(days=1), alpha=0.3, color=color)

    ax1.plot(df_2026.index, df_2026["NTU"].values, 'b-', linewidth=0.8, alpha=0.7)
    ax1.axhline(y=NTU_LIMIT, color='red', linestyle='--', label=f'NTU={NTU_LIMIT}')
    ax1.set_title("NTU时间序列与风险分级")
    ax1.set_ylabel("NTU")
    ax1.legend()

    # Add custom legend for risk levels
    from matplotlib.patches import Patch
    legend_elements = [
        Patch(facecolor='green', alpha=0.3, label='安全'),
        Patch(facecolor='yellow', alpha=0.3, label='低风险'),
        Patch(facecolor='orange', alpha=0.3, label='中风险'),
        Patch(facecolor='red', alpha=0.3, label='高风险'),
    ]
    ax1.legend(handles=[plt.Line2D([0], [0], color='b', linewidth=0.8, label='NTU'),
                        plt.Line2D([0], [0], color='red', linestyle='--', label=f'NTU={NTU_LIMIT}')] +
               legend_elements, loc='upper left')

    # 2. Risk level distribution pie chart
    ax2 = axes[0, 1]
    level_counts = [daily_df["risk_level"].value_counts().get(l, 0) for l in level_order]
    colors_pie = ['green', 'yellow', 'orange', 'red']
    ax2.pie(level_counts, labels=level_order, colors=colors_pie, autopct='%1.1f%%')
    ax2.set_title("风险等级分布")

    # 3. Risk score distribution
    ax3 = axes[1, 0]
    for i, level in enumerate(level_order):
        level_scores = daily_df[daily_df["risk_level"] == level]["risk_score"]
        ax3.hist(level_scores, bins=20, alpha=0.5, color=colors_pie[i], label=level)
    ax3.set_xlabel("风险分数")
    ax3.set_ylabel("频次")
    ax3.set_title("风险等级-分数分布")
```

```
    ax3.legend()

    # 4. Monthly stacked bar chart
    ax4 = axes[1, 1]
    months = sorted(daily_df["month"].unique())
    bar_data = {}
    for level in level_order:
        bar_data[level] = [daily_df[(daily_df["month"] == m) & (daily_df["risk_level"] == level)].shape[0]
                           for m in months]

    bottom = np.zeros(len(months))
    for i, level in enumerate(level_order):
        ax4.bar(months, bar_data[level], bottom=bottom, color=colors_pie[i],
                label=level, alpha=0.8)
        bottom += np.array(bar_data[level])
    ax4.set_xlabel("月份")
    ax4.set_ylabel("天数")
    ax4.set_title("月度风险等级分布")
    ax4.set_xticks(months)
    ax4.legend()
    ax4.grid(True, alpha=0.3, axis='y')

    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "problem4_risk_assessment.png"), dpi=150)
    plt.close()

    print(f"\nAll␣outputs␣saved␣to:␣{OUT_DIR}")
    print("Done!")


if __name__ == "__main__":
    main()
```

附录 B：参考文献

[nosep]

1. Friedman J H. Greedy function approximation: a gradient boosting machine[J]. Annals of Statistics, 2001, 29(5): 1189–1232.

2. Lundberg S M, Lee S I. A unified approach to interpreting model predictions[C]. NeurIPS, 2017: 4765–4774.

3. Cho K, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation[C]. EMNLP, 2014: 1724–1734.

4. Raissi M, Perdikaris P, Karniadakis G E. Physics-informed neural networks[J]. Journal of Computational Physics, 2019, 378: 686–707.

5. Shannon C E. A mathematical theory of communication[J]. Bell System Technical Journal, 1948, 27(3): 379–423.

6. Liu F T, Ting K M, Zhou Z H. Isolation forest[C]. ICDM, 2008: 413–422.

7. Dickey D A, Fuller W A. Distribution of the estimators for autoregressive time series with a unit root[J]. JASA, 1979, 74(366): 427–431.

8. 中华人民共和国卫生部. 生活饮用水卫生标准 (GB 5749-2022)[S]. 2022.